

Universidad Nacional de Córdoba



Facultad de Matemática, Astronomía, Física y Computación

Asignación Óptima de Viajes para Transporte de Maquinaria Agrícola con Restricciones Temporales y Operativas

por

Tomás Marmay

Presentado ante la **FACULTAD DE MATEMÁTICA, ASTRONOMÍA, FÍSICA Y COMPUTACIÓN** como parte de los requerimientos para la obtención del grado de Licenciado en Ciencias de la Computación de la

Universidad Nacional de Córdoba

Junio 2026

Director: Luis Ariel Biedma



Asignación Óptima de Viajes para Transporte de Maquinaria Agrícola con Restricciones Temporales y Operativas © 2026 by Tomas Marmay is licensed under Creative Commons Attribution 4.0 International.

Resumen

El Problema de Ruteo de Vehículos (VRP) es un problema de optimización combinatoria NP-Hard ampliamente estudiado por su relevancia en logística y distribución.

En este trabajo, se modela el problema del transporte de maquinaria agrícola en Argentina como una variante del VRP que incorpora múltiples depósitos, ventanas de tiempo, capacidad vehicular, demanda de clientes, costos fijos por vehículo y tiempos de servicio.

Se desarrollaron y compararon tres estrategias de resolución: una heurística basada en Nearest Neighbor (NN), un enfoque metaheurístico mediante Ant Colony Optimization (ACO) y un modelo de programación entera mixta (MILP).

Los experimentos se realizaron sobre instancias generadas para representar el dominio agrícola. Los resultados muestran que ACO obtiene soluciones con un costo cuatro veces menor al de la heurística base. Frente a los solvers MILP evaluados (Gurobi y HiGHS), ACO presenta un gap promedio de aproximadamente 33 %, pero lo alcanza en 80 segundos frente al límite de 1200 segundos asignado al MILP. Esto posiciona a ACO como una alternativa competitiva en contextos donde la velocidad de respuesta es prioritaria sobre la optimalidad exacta.

Keywords: VRP, ventanas de tiempo, múltiples depósitos, flota heterogénea, VRP abierto, heurística constructiva, metaheurística, ACO, MILP, transporte de maquinaria agrícola.

Abstract

The Vehicle Routing Problem (VRP) is an NP-Hard combinatorial optimization problem widely studied for its relevance in logistics and distribution.

In this work, the problem of agricultural machinery transport in Argentina is modeled as a variant of the VRP incorporating multiple depots, time windows, vehicle capacity, customer demand, fixed vehicle costs, and service times.

Three resolution strategies were developed and compared: a heuristic based on Nearest Neighbor (NN), a metaheuristic approach using Ant Colony Optimization (ACO), and a Mixed Integer Linear Programming (MILP) model.

Experiments were conducted on instances generated to represent the agricultural domain. Results show that ACO achieves solutions at four times lower cost than the baseline heuristic. Against the evaluated MILP solvers (Gurobi and HiGHS), ACO presents an average gap of approximately 33 %, but achieves this in 80 seconds compared to the 1200-second timeout assigned to MILP. This positions ACO as a competitive alternative in contexts where response time takes priority over exact optimality.

Keywords: VRP, time windows, multiple depots, heterogeneous fleet, Open VRP, constructive heuristic, metaheuristic, ACO, MILP, agricultural machinery transport.

Agradecimientos

A Luis Ariel Biedma, que cuando este trabajo todavía no era este trabajo me insistió en que apostara por el tema que realmente me interesaba. Gracias por esa convicción, por la dirección y por acompañar este proceso desde el principio hasta el final.

A la Universidad Nacional de Córdoba y a FaMAF, por ser el espacio donde todo esto fue posible. Me formaron tanto en lo profesional, como en lo personal.

A mis padres y mis abuelos, que hicieron todo lo posible para que yo pudiera llegar hasta acá. Cada esfuerzo silencioso, cada sacrificio que quizás nunca nombré pero siempre vi, está presente en cada página de este trabajo. Esta licenciatura también es de ustedes.

A mis amigos más cercanos, por estar siempre, incluso en los momentos en que desaparecía por estar estudiando.

Y a mí mismo: costó. Fue largo. Hubo momentos en que quise soltar. Pero la constancia ganó.

A todo aquel que se tome el tiempo de leer este trabajo: gracias.

Índice general

Resumen I

Abstract II

Agradecimientos III

1. Introducción **1**
1.1. Contexto y motivación . . . 1
1.2. Objetivos del trabajo . . . 1
1.3. Organización del documento . . . 2

2. Descripción del problema **3**
2.1. Literatura previa . . . 3
2.2. Definición general del problema . . . 4
2.3. Notación y formulación matemática . . . 4

3. Metodologías de resolución **8**
3.1. Time Window Nearest Neighbor (TWNN) . . . 8
3.1.1. Descripción . . . 8
3.1.2. Implementación . . . 9
3.2. Ant Colony Optimization (ACO) . . . 9
3.2.1. Descripción . . . 9
3.2.2. Implementación . . . 10
3.3. Programación Lineal Entera Mixta (MILP) . . . 14
3.3.1. Descripción . . . 14
3.3.2. Implementación . . . 14
3.3.3. Análisis de complejidad del modelo . . . 15

4. Experimentación numérica **18**
4.1. Generación de instancias . . . 18
4.2. Optimización de hiperparámetros . . . 19
4.3. Configuración experimental . . . 20
4.4. Infraestructura y reproducibilidad . . . 21

5. Resultados **22**
5.1. Criterios de evaluación . . . 22
5.2. Comparación . . . 22
5.2.1. Causas de las pérdidas de cobertura . . . 24
5.2.2. Calidad de las soluciones . . . 29

6. Conclusiones **37**
6.1. Líneas de trabajo futuro 37

A. Resultados completos de las ejecuciones **39**
A.1. TWNN 39
A.2. ACO 40
A.3. Gurobi 40
A.4. HiGHS 41

Bibliografía **43**

Índice de figuras

3.1. Ilustración del mecanismo de selección de caminos en ACO: los agentes construyen rutas guiados por las feromonas depositadas en las aristas y la atractividad heurística de cada nodo. 10

5.1. Cobertura por solver en función del número de clientes. 23
5.2. Comparación del flujo de ejecuciones entre MILP y ACO. 24
5.3. Uso de RAM por metodología. 25
5.4. Límite de RAM alcanzado. 26
5.5. Uso de CPU por metodología. 27
5.6. Escalabilidad del tiempo de cómputo por solver. 28
5.7. Perfil de rendimiento: fracción de instancias resueltas en función del tiempo de cómputo acumulado. 29
5.8. Distribución de costos de solución por solver. 30
5.9. Gap relativo al óptimo de Gurobi por solver. 31
5.10. Gap promedio de ACO respecto a Gurobi por configuración (c, v). 32
5.11. Evolución del costo de ACO en la instancia de mayor gap ($c = 50$, $v = 25$). . . 33
5.12. Fracción de vehículos utilizados por solver. 34
5.13. Visualización de asignaciones de clientes a rutas. 35

Índice de cuadros

3.1. Crecimiento de cada familia de restricciones del modelo MILP. 16

4.1. Efecto de `tw_tightness` sobre las ventanas generadas ($T_{\max} = 49$, semilla 42, $|C| = 3$, $|V| = 2$). Con `tw_tightness`$= [0, 500]$ las ventanas se expanden hasta cubrir el horizonte completo, eliminando en la práctica las restricciones temporales. 19
4.2. Ventanas de tiempo generadas por el generador aleatorio ($T_{\max} = 32$, semilla 42, $|C| = 3$, $|V| = 2$). 19
4.3. Parámetros fijos del experimento. 20
4.4. Parámetros fijos de ACO. 20

5.1. Estadísticos del gap respecto a Gurobi. 31

A.1. Resultados de TWNN sobre las 30 instancias. 39
A.2. Resultados de ACO sobre las 30 instancias. 40
A.3. Resultados de Gurobi sobre las 30 instancias. 41
A.4. Resultados de HiGHS sobre las 30 instancias. 41

Capítulo 1

Introducción

1.1. Contexto y motivación

El sector agropecuario es uno de los pilares fundamentales de la economía argentina. Su participación en las exportaciones nacionales, su rol como generador de empleo en las regiones del interior del país y su peso en el producto bruto interno lo posicionan como un sector estratégico para el desarrollo económico del país.

Dentro de este sector, el transporte de maquinaria agrícola representa una operación cotidiana pero crítica: cuando un productor adquiere maquinaria nueva o necesita trasladarla entre sus campos, o cuando contrata a un tercero para realizar servicios de maquinaria, surge el problema de encontrar un transportista disponible y a un precio razonable. Hoy en día, esto se resuelve de forma manual: el productor contacta a un conjunto reducido de empresas transportistas conocidas en la zona y solicita cotizaciones individuales. El costo de cada viaje incluye tanto el trayecto de ida (cargado con la maquinaria) como el trayecto de vuelta vacío, ya que el transportista debe retornar a su base de operaciones. Como no existe ningún sistema centralizado que conozca la ubicación y disponibilidad de los transportistas en tiempo real, esos viajes de vuelta vacíos no pueden ser aprovechados: un transportista que acaba de descargar maquinaria cerca del campo de otro cliente potencial no tiene forma de saberlo, y ese viaje se pierde como costo muerto.

Esta ineficiencia sistémica es análoga a la que existía en otros sectores antes de la aparición de plataformas de coordinación: así como PedidosYa redujo los costos logísticos del reparto de comida, o Uber optimizó la asignación de vehículos de transporte de pasajeros, el sector del transporte de maquinaria agrícola carece hoy de un sistema equivalente que permita minimizar los costos de manera coordinada. Construir ese sistema requiere, como base, un modelo matemático capaz de tomar decisiones de asignación y ruteo de forma óptima. La presente tesis aborda precisamente ese problema.

1.2. Objetivos del trabajo

El presente trabajo tiene los siguientes objetivos:

1. Revisar la literatura existente sobre el problema de ruteo de vehículos y sus variantes.
2. Modelar formalmente el problema de transporte de maquinaria agrícola, identificando sus características operativas y derivando una variante adecuada del problema de ruteo de vehículos.

3. Implementar tres estrategias de resolución de distinta naturaleza y complejidad computacional.
4. Desarrollar un *framework* de experimentación reproducible que permita comparar las estrategias bajo condiciones controladas.
5. Calibrar los hiperparámetros de las estrategias de resolución implementadas.
6. Diseñar y ejecutar experimentos sobre instancias de distintos tamaños.
7. Analizar y comparar los resultados obtenidos en términos de calidad de solución y tiempo de cómputo.

1.3. Organización del documento

El resto del documento se organiza de la siguiente manera. El Capítulo 2 presenta la descripción del problema: primero se revisa la literatura existente sobre el ruteo de vehículos y sus principales variantes, contextualizando la elección del modelo adoptado; a continuación se introduce la formulación matemática. El Capítulo 3 describe las tres estrategias de resolución implementadas, en orden creciente de complejidad, e incluye un análisis del crecimiento del modelo exacto en función del tamaño de la instancia. El Capítulo 4 detalla el diseño experimental: la generación de instancias, los parámetros utilizados, la calibración de hiperparámetros y la infraestructura que garantiza la reproducibilidad de los resultados. El Capítulo 5 analiza los resultados obtenidos y presenta la comparación entre estrategias. Finalmente, el Capítulo 6 presenta las conclusiones del trabajo y las líneas de investigación futura.

Capítulo 2

Descripción del problema

2.1. Literatura previa

Los Problemas de Ruteo de Vehículos (VRP) han sido intensamente estudiados durante los últimos 60 años. El VRP con Capacidad fue introducido por Dantzig y Ramser en 1959 [1], bajo el nombre de "Problema de Despacho de Camiones", como una generalización del Problema del Viajante de Comercio (TSP) al incorporar múltiples vehículos. Desde entonces, se desarrollaron numerosas variantes con el objetivo de modelar problemas reales de distribución con mayor fidelidad.

El TSP pertenece a la clase NP-Hard [2] debido a que el número de rutas posibles crece factorialmente con la cantidad de ciudades (hay $(n-1)!/2$ rutas distintas para n ciudades), lo que hace que la búsqueda exhaustiva sea computacionalmente inviable incluso para instancias de tamaño moderado, y no se conoce ningún algoritmo que lo resuelva en tiempo polinómico. El VRP, que contiene al TSP como caso particular, es también NP-Hard; las distintas variantes que incorporan restricciones adicionales (capacidad, ventanas de tiempo, múltiples depósitos) extienden aún más esta complejidad. Esta característica tiene consecuencias prácticas directas: para instancias de tamaño real, los métodos exactos se vuelven computacionalmente inabordables, lo que motiva el uso de heurísticas y metaheurísticas capaces de obtener soluciones de buena calidad en tiempos razonables.

Las metaheurísticas son métodos de búsqueda iterativa que exploran el espacio de soluciones de forma guiada sin garantía de optimalidad. En cada iteración parten de una solución existente, generan candidatos en su vecindad y los evalúan según una función de costo, aceptando o rechazando cada candidato según su desempeño. El proceso se repite hasta cumplir algún criterio de parada, como un número máximo de iteraciones o la convergencia de la solución. Entre los algoritmos metaheurísticos más utilizados en problemas de ruteo se encuentran los algoritmos genéticos, la búsqueda tabú y Ant Colony Optimization (ACO), este último inspirado en el comportamiento colectivo de las colonias de hormigas y descripto en detalle en la Sección 3.2.

La aplicación del VRP en contextos agrícolas ha sido explorada desde distintos ángulos. Bochtis y Sørensen [20] modelan las tareas de planificación y programación de las unidades de servicio en operaciones agrícolas de campo como instancias del VRPTW, donde las unidades de servicio actúan como vehículos y las unidades primarias como clientes. Bakhtiari et al. [21] utilizan ACO para resolver el problema de planificar rutas óptimas de cobertura de campo en operaciones agrícolas, con el objetivo de minimizar recorridos improductivos cumpliendo con los criterios operativos del dominio. Gracia et al. [17] formulan el problema de recolección de fardos post-cosecha como un problema de optimización de rutas y lo resuelven mediante un algoritmo genético híbrido. Durante la investigación no se encontraron trabajos previos que

aborden el problema de asignación óptima entre vehículos y clientes en el contexto agrícola. Dado que el framework desarrollado en este trabajo no impone restricciones específicas a ese dominio, la búsqueda de antecedentes puede extenderse a la literatura general de VRP, donde el concepto de cliente es abstracto y no está ligado a ningún caso de uso particular.

2.2. Definición general del problema

Para modelar el problema del transporte de maquinaria agrícola con fidelidad, es necesario ir más allá del VRP base e incorporar restricciones que reflejan la realidad operativa del sector. En el campo, distintos tipos de maquinaria requieren distintos tipos de vehículo de transporte, y las necesidades de traslado están sujetas a fechas y ventanas horarias determinadas por el ciclo productivo. Los vehículos no suelen ser propiedad de los productores sino de terceros que operan bajo demanda; cada tipo de vehículo maneja un costo fijo diferente, ya que los transportistas cotizan por arranque más precio por kilómetro, lo que motiva minimizar los kilómetros recorridos. Además, los transportistas harán todo lo posible por aprovechar el trayecto de vuelta, incluso encadenando un segundo viaje si hay demanda disponible en la zona de destino. Estas características dan lugar a una variante del VRP con múltiples depósitos, ventanas de tiempo, flota heterogénea y rutas abiertas, que se describe a continuación.

El VRP base dio lugar a numerosas variantes que incorporan restricciones adicionales para modelar con mayor fidelidad los problemas reales de distribución [18]. Una de las más estudiadas es el Capacitated VRP (CVRP, VRP con Capacidad), presente ya en el trabajo original de Dantzig y Ramser [1] y desarrollado operativamente por Clarke y Wright [15], que limita la demanda acumulada de cada ruta a la capacidad del vehículo asignado. El Vehicle Routing Problem with Time Windows (VRPTW, VRP con Ventanas de Tiempo) [12] agrega la restricción de que cada cliente debe ser atendido dentro de un intervalo de tiempo prefijado; Kallehauge et al. [3] presentan una formulación matemática completa de esta variante ampliamente adoptada en la literatura. Cuando los vehículos se distribuyen entre varios puntos de origen en lugar de uno solo, se obtiene el Multi-Depot VRP (MDVRP, VRP con Múltiples Depósitos); su combinación con ventanas de tiempo da lugar al MDVRPTW [19].

El problema abordado en este trabajo extiende el MDVRPTW con tres restricciones adicionales. La flota es heterogénea: cada vehículo tiene su propia capacidad Q_k y costo fijo F_k, lo que lo enmarca dentro de los problemas de *Fleet Size and Mix* (Tamaño y Composición de Flota), una clase de problemas en la que el número y el tipo de vehículos a utilizar también forman parte de la decisión de optimización. El tamaño de la flota activa es también una variable de decisión: incorporar un vehículo a la solución implica un costo fijo F_k, por lo que el modelo determina cuántos y cuáles vehículos conviene utilizar. Por último, el nodo de retorno Ret_k puede diferir del depósito de origen, clasificando las rutas como abiertas (Open VRP). La combinación de todas estas restricciones corresponde a lo que en la literatura se denomina HFV-MD-OVRPTW (*Heterogeneous Fixed Plus Variable Cost Multi-Depot Open VRP with Time Windows*), aunque no existe un nombre universalmente establecido para esta variante específica.

2.3. Notación y formulación matemática

A fin de representar el problema de manera formal, se define a continuación la notación y la formulación matemática del problema.

La notación utilizada a lo largo de este documento:

- V: conjunto de vehículos

- C: conjunto de clientes
- N: conjunto de nodos
- d_{ij}: matriz de costo del nodo i al nodo j
- F_k: penalización por agregar el vehículo k
- $Depot_k$: depósito del vehículo k
- Ret_k: nodo de retorno del vehículo k
- Q_k: capacidad del vehículo k
- D_i: demanda del cliente i
- t_{ij}: tiempo de viaje desde i hasta j
- $[e_i,\, l_i]$: ventana de tiempo para el nodo i
- s_i^k: tiempo de servicio del cliente i con el vehículo k
- $x_{ij}^k = \begin{cases} 1 & \text{si el vehículo } k \text{ viaja de } i \text{ a } j \\ 0 & \text{en caso contrario} \end{cases}$
- $u^k \in \{0, 1\}$: variable binaria para la selección del vehículo k

Con esta notación, podemos presentar el problema de la siguiente manera:

$$\text{mín} \quad \sum_{k\in V}\sum_{i\in N}\sum_{j\in N} d_{ij}\, x_{ij}^{k} + \sum_{k\in V} F_k\, u^k \tag{2.1}$$

$$\text{s.t.} \quad \sum_{k\in V}\sum_{j\in N} x_{ij}^{k} = 1 \quad \forall\, i \in C \tag{2.2}$$

$$\sum_{i\in C} D_i \sum_{j\in N} x_{ij}^{k} \le Q_k \quad \forall\, k \in V \tag{2.3}$$

$$\sum_{j\in N} x_{Depot_k,\, j}^{k} = u^k \quad \forall\, k \in V \tag{2.4}$$

$$\sum_{i\in N} x_{ih}^{k} - \sum_{j\in N} x_{hj}^{k} = 0 \quad \forall\, h \in C,\ \forall\, k \in V \tag{2.5}$$

$$\sum_{i\in N} x_{i,\, Ret_k}^{k} = u^k \quad \forall\, k \in V \tag{2.6}$$

$$x_{ij}^{k}\left(s_i^k + t_{ij} - s_j^k\right) \le 0 \quad \forall\, i,j \in N,\ \forall\, k \in V \tag{2.7}$$

$$e_i \le s_i^k \le l_i \quad \forall\, i \in N,\ \forall\, k \in V \tag{2.8}$$

$$x_{ij}^{k} \in \{0,1\} \quad \forall\, i,j \in N,\ \forall\, k \in V \tag{2.9}$$

$$s_i^k \in R \quad \forall\, i \in N,\ \forall\, k \in V \tag{2.10}$$

$$u^k \in \{0,1\} \quad \forall\, k \in V \tag{2.11}$$

$$\sum_{i\in N}\sum_{j\in N} x_{ij}^{k} \le N \cdot u^k \quad \forall\, k \in V \tag{2.12}$$

$$\sum_{k\in V} u^k \ge 1 \tag{2.13}$$

La función objetivo (2.1) combina dos términos. El primero, $\sum_{k\in V}\sum_{i\in N}\sum_{j\in N} d_{ij}\, x_{ij}^{k}$, minimiza los kilómetros totales recorridos por la flota: dado que los transportistas cobran por kilómetro, reducir la distancia total impacta directamente en el costo del servicio y adicionalmente reduce el desgaste de neumáticos y las emisiones. El segundo término, $\sum_{k\in V} F_k\, u^k$, penaliza la activación de cada vehículo con un costo fijo F_k, que representa el arranque cobrado por el transportista independientemente de la distancia recorrida; minimizarlo desincentiva contratar vehículos innecesarios.

Cabe señalar que el modelo no contempla algunos aspectos que podrían incorporarse en trabajos futuros: el precio por kilómetro de cada transportista en particular (reemplazando d_{ij} por $d_{ij} \cdot p_k$, donde p_k es el precio por kilómetro del vehículo k), ni criterios de equidad en la distribución de viajes entre transportistas. Ambas extensiones son compatibles con la formulación presentada y se dejan como líneas de trabajo futuro.

La restricción (2.2) asegura que los clientes son visitados exactamente una vez y (2.3) limita la carga de cada vehículo hasta capacidad máxima.

Las ecuaciones (2.4), (2.5) y (2.6) garantizan la coherencia del flujo: cada vehículo activo debe partir de su depósito, conservar el flujo de entrada y salida en cada cliente intermedio, y finalizar en su nodo de retorno.

La restricción (2.7) garantiza la propagación correcta de los tiempos de servicio: si el vehículo k viaja de i a j, el tiempo de inicio en j debe ser al menos el tiempo de inicio en i más el tiempo de viaje. Para su implementación en el modelo de Mixed Integer Linear Programming (MILP, Programación Lineal Entera Mixta), la restricción (2.7) se linealiza mediante la técnica Big-M,

una estrategia estándar para eliminar productos entre variables en programación lineal: se reemplaza la no linealidad por una constante suficientemente grande M_{ij} que hace la restricción inactiva cuando la condición no aplica. En este caso $M_{ij} = l_i + t_{ij} - e_j$.

La restricción (2.8) asegura que el vehículo llegue a cada cliente dentro de su ventana horaria: no puede llegar antes de que el cliente esté disponible ni después de que cierre. Las restricciones (2.9) y (2.11) establecen que las variables de decisión son binarias: un vehículo o hace el recorrido de i a j o no lo hace, y un vehículo o es activado o no lo es; no hay valores intermedios. La restricción (2.10) indica que los tiempos de inicio de servicio son variables continuas, ya que un vehículo puede llegar a cualquier hora dentro de la ventana, no necesariamente en valores enteros.

Es necesario agregar (2.12) para asegurar la coherencia entre las variables x_{ij}^k y u^k, de modo que un vehículo sólo pueda realizar desplazamientos si ha sido activado en la solución.

La formulación presentada en las ecuaciones (2.1)–(2.10) se toma de Kallehauge et al. [3], que describe el modelo base. En este trabajo dichas restricciones fueron ligeramente modificadas para incorporar las variables u^k, y adaptar el modelo al caso de múltiples depósitos y retornos. Por otro lado, las restricciones (2.11)–(2.13) fueron desarrolladas por el autor para ajustar el modelo a las particularidades del problema estudiado.

Si bien el modelo presentado no incorpora explícitamente las restricciones del *Pickup and Delivery Problem* (PDP), puede extenderse a ese caso mediante una reinterpretación de los datos de entrada. Cada cliente i posee un punto de recogida $origin_i$ y uno de entrega $destination_i$. Bajo el supuesto de que el vehículo completa el ciclo $origin_i \rightarrow destination_i$ antes de dirigirse al próximo cliente, la carga nunca se acumula simultáneamente y el grafo de decisión no cambia. En ese caso basta con redefinir $c_{ij} = c_{destination_i, origin_j} + c_{origin_j, destination_j}$ y $t_{ij} = t_{destination_i, origin_j}$, e incorporar $t_{origin_i, destination_i}$ al tiempo de servicio en i. Con estas redefiniciones el modelo resuelve el PDP sin modificar ninguna restricción.

Se optó por no modelar el PDP en su forma general para no agregar complejidad innecesaria al problema. Los supuestos adoptados son válidos en el contexto del transporte de maquinaria agrícola: la entrega es secuencial, no existen ventanas de tiempo en $destination_i$, y no se contempla la posibilidad de transportar pedidos de múltiples clientes de forma simultánea. Extender el modelo al caso general, donde estos supuestos se relajan y se requieren restricciones explícitas de precedencia y acoplamiento, queda como línea de trabajo futuro.

Capítulo 3

Metodologías de resolución

Los problemas de ruteo de vehículos pueden abordarse desde distintos enfoques según el balance deseado entre calidad de solución y tiempo de cómputo. En un extremo, los métodos exactos garantizan encontrar la solución óptima del modelo, pero su costo computacional los hace inviables para instancias de gran tamaño. En el otro extremo, las heurísticas constructivas sacrifican optimalidad a cambio de soluciones rápidas y fáciles de implementar. Entre ambos extremos se ubican las metaheurísticas, que exploran el espacio de soluciones de forma inteligente y obtienen soluciones de buena calidad en tiempos razonables sin garantía de optimalidad.

En este trabajo se implementaron tres estrategias que cubren ese espectro deliberadamente: una heurística constructiva simple (TWNN) que sirve como línea de base y permite evaluar cuánto aportan los enfoques más sofisticados; una metaheurística (ACO) que representa el balance entre calidad y tiempo de cómputo; y un modelo exacto (MILP) que actúa como referencia de optimalidad para instancias donde la resolución exacta es viable. Las tres estrategias se describen a continuación en orden creciente de complejidad.

3.1. Time Window Nearest Neighbor (TWNN)

3.1.1. Descripción

Este algoritmo, denominado *Time Window Nearest Neighbor* (TWNN) a lo largo de este trabajo, nace de la necesidad de un baseline simple que sirva como referencia para comparar con enfoques de mayor complejidad computacional. No busca optimizar exhaustivamente el costo total, sino construir rápidamente una solución factible que respete todas las restricciones del problema. Este tipo de heurísticas constructivas basadas en el vecino más cercano es ampliamente utilizado en la literatura como cota de referencia para evaluar la eficiencia de enfoques más sofisticados: por ejemplo, [17] emplea una heurística de vecino más cercano como cota superior para evaluar la eficiencia de un algoritmo genético híbrido aplicado al problema de recolección de fardos (BCP), una variante del VRP con capacidad orientada a maquinaria agrícola.

La heurística se inspira en una estrategia de planificación humana natural: priorizar clientes con mayor urgencia temporal y, una vez iniciada una ruta, seleccionar secuencialmente el cliente factible más cercano. Desde el punto de vista algorítmico, el método puede interpretarse como una heurística constructiva *greedy* en dos fases: una asignación inicial orientada a la factibilidad temporal y una construcción de rutas basada en una variante del algoritmo Nearest Neighbor adaptada a ventanas de tiempo.

3.1.2. Implementación

La implementación de este algoritmo consta de dos fases bien diferenciadas. En la primera fase se ordenan los clientes según el centro de su ventana temporal, definido como:

$$\frac{e_i + l_i}{2}$$

Este criterio prioriza a los clientes con menor margen de atención.

Siguiendo dicho orden, se asigna cada cliente al vehículo que minimiza el tiempo de inicio de servicio: máx(s_i, e_i), considerando únicamente aquellos vehículos para los cuales dicho tiempo es factible (es decir, máx$(s_i, e_i) \leq l_i$) y que aún cuenten con capacidad disponible.

Si ningún vehículo puede atender a un cliente respetando su ventana de tiempo, la instancia se declara infactible. Esta fase finaliza con una asignación cliente-vehículo que determina qué clientes serán atendidos por cada vehículo.

En la segunda fase se construye la ruta de cada vehículo. Partiendo desde su depósito de origen, se selecciona iterativamente el cliente aún no visitado con menor tiempo de viaje desde el nodo actual, siempre que su atención sea factible dentro de su ventana de tiempo. Este proceso se repite hasta que todos los clientes asignados al vehículo han sido visitados, momento en el que la ruta se cierra dirigiéndose al nodo de retorno correspondiente.

3.2. Ant Colony Optimization (ACO)

3.2.1. Descripción

Ant Colony Optimization (ACO), u optimización por colonia de hormigas, es una metaheurística cuya formulación fue consolidada por Marco Dorigo y Thomas Stützle [16], inspirada en el comportamiento de las colonias de hormigas. Pertenece a la familia de la *swarm intelligence* (inteligencia de enjambre), un área de la inteligencia artificial y la computación bioinspirada que modela el comportamiento colectivo de grupos de animales, como hormigas, abejas, pájaros o peces, donde individuos simples, sin un líder central, colaboran para resolver problemas complejos de manera eficiente.

Las hormigas exploran aleatoriamente los alrededores del hormiguero. Cuando encuentran comida, regresan dejando un rastro de feromonas por el camino de vuelta, una sustancia que puede ser detectada por otras hormigas.

Las hormigas pueden seguir varios caminos hacia la comida y luego de vuelta hacia el hormiguero. Se observó que, gracias al refuerzo del rastro de feromonas de las sucesivas hormigas que viajan desde la comida al hormiguero, solo el camino más corto se mantiene en uso, dado que las hormigas prefieren seguir el camino con mayor concentración de feromonas. El refuerzo de las feromonas es autocatalítico (es decir, el propio proceso se refuerza a sí mismo): mientras más corto sea el camino, menor tiempo le tomará a las hormigas ir y volver, reforzando la feromona en el mejor trayecto, mientras que las hormigas en trayectos más largos siguen aún en viaje.

A su vez, las feromonas se evaporan gradualmente con el tiempo: esto evita que el sistema converja prematuramente hacia una solución subóptima y mantiene la capacidad de exploración del algoritmo.

Luego de un tiempo, la mayoría de las hormigas termina usando el camino más corto, mientras que una minoría opta por caminos alternativos. El comportamiento de esta minoría es importante, ya que permite explorar nuevos caminos que pueden derivar en mejores rutas no

consideradas inicialmente. Este balance entre explotar los caminos conocidos y explorar nuevos constituye un principio central en el diseño de metaheurísticas, conocido como el dilema exploración–explotación.

ACO replica este mecanismo, añadiendo mejoras para hacerlo más eficiente en su implementación. Las hormigas son reemplazadas por un conjunto de agentes artificiales cuyo objetivo individual es construir soluciones visitando nodos de un grafo, donde cada nodo representa un cliente, un depósito o un nodo de retorno, y cada arista representa el trayecto entre dos de ellos con su costo asociado. Situado en un nodo, cada agente tiene en cuenta dos parámetros para decidir cuál es el siguiente nodo: las feromonas y el atractivo. La elección del siguiente nodo es siempre probabilística, guiada por ambos parámetros: en la mayoría de los casos el agente seguirá las opciones con mayor concentración de feromonas y mayor atractivo, pero la naturaleza estocástica del proceso permite que, ocasionalmente, se exploren caminos menos transitados, tal como ilustra la Figura 3.1.



Figura 3.1: Ilustración del mecanismo de selección de caminos en ACO: los agentes construyen rutas guiados por las feromonas depositadas en las aristas y la atractividad heurística de cada nodo.

3.2.2. Implementación

La implementación utiliza dos matrices de feromonas que aprenden aspectos complementarios del problema:

- τ_{vc}^{asig}, con $v \in V$ y $c \in C$: registra la afinidad de asignar el cliente c al vehículo v.
- τ_{ij}^{ruta}, con $i, j \in N$: registra la afinidad de visitar el nodo j inmediatamente después del nodo i dentro de una ruta.

Ambas se inicializan en $\tau_{\text{mín}}$.

Fase 1 — Asignación cliente-vehículo

Cada hormiga asigna cada cliente c a un vehículo. La *atractividad heurística* de asignar c al vehículo v se define como el inverso del costo entre el depósito de v y c:

$$\eta_{vc} = \frac{1}{d[\text{Depot}_v][c]}$$

La probabilidad de asignación es:

$$P(c \to v) = \frac{(\tau_{vc}^{asig})^{\alpha} \cdot (\eta_{vc})^{\beta}}{\sum_{v' \in V} (\tau_{v'c}^{asig})^{\alpha} \cdot (\eta_{v'c})^{\beta}}$$

donde α pondera la información histórica (feromonas) y β la información heurística.

Fase 2 — Construcción de rutas

A partir de la asignación obtenida, se construye la ruta de cada vehículo mediante una selección de tipo ACS (*Ant Colony System*): con probabilidad q_0 se elige el cliente con mayor puntuación (*explotación*); de lo contrario, se realiza una selección probabilística por ruleta (*exploración*).

La puntuación de visitar el cliente j desde el nodo actual i combina tres componentes:

$$\text{score}(i,j) = \left(\tau_{ij}^{ruta}\right)^{\alpha} \cdot \eta_{ij}^{\beta} \cdot \mu_{ij}^{\gamma}$$

donde:

- $\eta_{ij} = \dfrac{1}{t_{ij} + W_u \cdot s_j}$ penaliza el tiempo de viaje t_{ij} y la urgencia del cliente j, medida por su tiempo de inicio de servicio $s_j = \text{máx}(\text{llegada}_j,\ e_j)$.
- $\mu_{ij} = \text{máx}(0,\ d_{i,\text{dep}} + d_{\text{dep},j} - d_{ij}) + 1$ es el ahorro de Clarke-Wright [15]: cuánto se reduce el costo yendo directamente de i a j en lugar de hacer el desvío por el depósito. Un ahorro elevado indica que i y j están lejos del depósito pero cerca entre sí, lo que los convierte en un buen par para visitar de forma consecutiva.
- γ es el exponente que controla el peso relativo del ahorro de Clarke-Wright en la puntuación final.

Solo son candidatos los clientes j para los cuales la llegada es factible ($\text{llegada}_j \leq l_j$) y el vehículo puede retornar a tiempo tras visitarlos.

Fase 3 — Actualización de feromonas

Se selecciona la mejor solución de la iteración y se aplica **evaporación** a ambas matrices:

$$\tau_{vc}^{asig} := \tau_{vc}^{asig} \cdot (1-\rho) \qquad \tau_{ij}^{ruta} := \tau_{ij}^{ruta} \cdot (1-\rho)$$

donde $\rho \in (0,\ 1]$ es la tasa de evaporación: controla qué fracción de la feromona se disipa en cada iteración, regulando así la velocidad con que el algoritmo olvida información histórica.

Luego se **refuerzan** las asignaciones y los arcos de la mejor solución con Δ:

$$\Delta := \begin{cases} \dfrac{\Phi}{\text{mejor costo}} & \text{si la solución es factible} \\ \dfrac{\Phi}{\text{mejor costo} + \lambda \cdot |\text{clientes no visitados}|} & \text{si la solución es infactible} \end{cases}$$

donde Φ es una constante de escala que determina la magnitud del refuerzo, y λ es el peso de penalización aplicado por cada cliente no atendido en soluciones infactibles.

Ambas matrices se recortan al rango $[\tau_{\text{mín}}, \tau_{\text{máx}}]$ tras cada actualización, donde $\tau_{\text{mín}}$ y $\tau_{\text{máx}}$ son los límites inferior y superior que evitan que las feromonas colapsen a cero o dominen completamente la exploración.

Post-procesamiento

Al finalizar el ciclo de iteraciones, la mejor solución se refina en dos pasos:

1. **Eliminación de vehículos.** Se intenta reasignar los clientes de un vehículo al resto mediante inserción por menor incremento de costo, respetando las ventanas de tiempo. Si la reasignación es factible y no incrementa el costo total, el vehículo es eliminado. El proceso se repite hasta que ningún vehículo pueda eliminarse.

2. **Búsqueda local 2-opt intra-ruta.** Para cada ruta, se invierten segmentos de forma iterativa mientras la nueva secuencia sea factible y reduzca el costo.

En cada iteración se lanza un conjunto de `num_ants` hormigas, cada una construyendo una solución completa de forma independiente. El criterio de parada es un límite de tiempo de 80 segundos por instancia. Los valores de los parámetros

$$(\tau_{\text{mín}}, \tau_{\text{máx}}, \alpha, \beta, \rho, \Phi, \lambda, \gamma, W_u, q_0, \texttt{num_ants})$$

se discuten en la Sección 4.2.

El Algoritmo 1 presenta el pseudocódigo de la implementación.

Algorithm 1 ACO v2 para MD-VRP-TW

Inicializar $\tau_{vc}^{asig} := \tau_{\text{mín}}, \quad \tau_{ij}^{ruta} := \tau_{\text{mín}}$
while tiempo $<$ límite **do**
  for cada hormiga **do**
    Fase 1 — Asignación:
    for cada cliente c **do**
      Calcular $P(c \to v)$ con τ^{asig}; asignar c al vehículo v
    end for
    Fase 2 — Construcción de rutas:
    for cada vehículo v **do**
      while hay clientes pendientes y factibles **do**
        Calcular score(i, j) para cada candidato j
        if rand() $< q_0$ **then**
          Elegir $j^* = \arg\max_j$ score(i, j) ▷ *explotación*
        else
          Elegir j por ruleta sobre score ▷ *exploración*
        end if
      end while
    end for
  end for
  Seleccionar mejor solución de la iteración
  Evaporar y reforzar τ^{asig} y τ^{ruta} con Δ
end while
Aplicar eliminación de vehículos a la mejor solución
Aplicar 2-opt intra-ruta a cada ruta
return mejor solución

Evolución del diseño

La implementación descripta es el resultado de un proceso iterativo. La versión inicial contaba con una sola matriz de feromonas (τ^{asig}) y una construcción de rutas puramente greedy. En esa versión, el gap (diferencia porcentual entre el costo de la solución encontrada y el óptimo de referencia) respecto a las soluciones de MILP era de aproximadamente el 100 %. Al analizar soluciones puntuales, se observó que ACO asignaba significativamente más vehículos que MILP: la fase de asignación seguía un criterio greedy por capacidad análogo al de TWNN, por lo que mejoraba marginalmente sobre esa heurística pero quedaba estructuralmente limitada sin ningún mecanismo que permitiera redistribuir clientes entre vehículos ya asignados. Esto motivó la incorporación de la segunda matriz τ^{ruta}, la selección ACS y el paso de eliminación de vehículos, logrando una mejora drástica en el desempeño.

La búsqueda local implementada opera a nivel **intra-ruta**: invierte segmentos dentro de una misma ruta. Una extensión natural sería el 2-opt **inter-ruta**, que permitiría intercambiar clientes entre rutas distintas y reducir así la brecha remanente con el óptimo de MILP. Su implementación se deja como trabajo futuro.

3.3. Programación Lineal Entera Mixta (MILP)

3.3.1. Descripción

La programación lineal (LP), introducida por George Dantzig en 1947 [4], permitió formular y resolver de manera sistemática una amplia clase de problemas de optimización bajo restricciones lineales. Khachiyan [22] demostró que LP pertenece a la clase P, es decir, existe un algoritmo que lo resuelve en tiempo polinomial, lo que lo convierte en un problema tratable en la práctica. Sin embargo, muchos problemas reales requieren decisiones discretas: si un vehículo hace o no un recorrido, si es activado o no. Al incorporar variables binarias o enteras se obtiene la Programación Lineal Entera Mixta (MILP) [5], que puede representar estas decisiones con exactitud pero a un costo computacional significativamente mayor: un MILP es en general NP-Hard, ya que el solver debe explorar un árbol de decisiones combinatorio mediante *branch and bound*, cuyo tamaño crece exponencialmente con el número de variables enteras.

A diferencia de las heurísticas y metaheurísticas, el enfoque MILP garantiza optimalidad: la solución obtenida es la óptima global del modelo formulado. Sin embargo, esta garantía tiene un costo computacional significativo. De forma intuitiva, los factores principales son:

- El número de variables binarias x_{ij}^k crece en orden $\mathcal{O}(|V|\cdot|N|^2)$ donde $|N| = |C|+2|V|$. Las variables auxiliares u^k y s_i^k añaden dimensiones adicionales al espacio de búsqueda.
- Las restricciones de flujo, capacidad y ventanas de tiempo incrementan el número de ecuaciones y la conectividad entre variables, aumentando la dificultad del proceso de ramificación y acotamiento (*branch and bound*) que utilizan los solvers exactos.

Estos factores implican un crecimiento exponencial del tiempo de cómputo en función del tamaño de la instancia: duplicar el número de clientes o de vehículos puede aumentar drásticamente el tiempo de resolución.

Existen estrategias para mitigar estas limitaciones. Kallehauge et al. [3] documentan, en el contexto de métodos exactos para el VRPTW, mejoras atribuibles al preprocesamiento y la poda de arcos infactibles según ventanas de tiempo y capacidades, a la eliminación de columnas inactivas durante el proceso de generación de columnas, al parado temprano del subproblema de precios en cuanto se genera una columna de costo reducido negativo, a la incorporación de *2-path cuts* para reducir la brecha de integralidad, y a estrategias de *branching* sobre ventanas de tiempo. Sin embargo, su implementación excede los objetivos de este trabajo.

3.3.2. Implementación

Para la implementación del modelo se utilizó el *framework* Pyomo [6, 7], que permite codificar las restricciones detalladas en la Sección 2.3 de forma declarativa e independiente del solver subyacente. Una vez definido el modelo, la pregunta natural es quién lo resuelve: ahí es donde entra la elección del solver.

En el ecosistema MILP existen dos grandes familias: solvers open source y solvers comerciales. Esta distinción no es menor, ya que afecta directamente la reproducibilidad de los resultados, el costo de adopción y el rendimiento en instancias de gran tamaño.

Dentro de los solvers open source, HiGHS [8] representa hoy la referencia del ecosistema. Desarrollado en la Universidad de Edinburgh, es de uso completamente gratuito y se encuentra en activo desarrollo. Su competitividad en instancias medianas lo convierte en una opción sólida para trabajos de investigación, con la ventaja adicional de que cualquier lector puede reproducir los resultados sin necesidad de licencia comercial.

En el extremo opuesto, Gurobi [9] es considerado el solver comercial de referencia en rendimiento. En instancias grandes o con restricciones temporales estrictas, la diferencia respecto a los solvers open source puede ser de órdenes de magnitud. Si bien requiere licencia comercial en entornos productivos, ofrece una licencia académica gratuita que lo hace accesible en el contexto universitario.

En la misma familia open source se encuentran CBC [10] (*COIN-OR Branch and Cut*), desarrollado por John Forrest a comienzos de los años 2000, y GLPK [11] (*GNU Linear Programming Kit*), desarrollado por Andrew Makhorin en 2001.

El análisis comparativo de este trabajo se centra en HiGHS y Gurobi. Adicionalmente, se evaluaron CBC y GLPK como alternativas: si bien resolvieron instancias de pequeño tamaño, su rendimiento se degradó significativamente en instancias medianas y grandes, por lo que fueron descartados del análisis principal. Pyomo hace posible este tipo de comparación sin modificar una sola línea del modelo, simplemente cambiando el solver configurado.

En todos los casos se utilizó la configuración por defecto que Pyomo aplica a cada solver, sin ajustes adicionales de parámetros internos. Explorar configuraciones personalizadas orientadas a mejorar el rendimiento en instancias de mayor tamaño se deja como trabajo futuro.

Para permitir la comparación en instancias donde la resolución exacta sería computacionalmente prohibitiva, se utiliza el mecanismo de timeout de Pyomo. Los solvers MILP resuelven el problema mediante *branch and bound* (B&B): un algoritmo que explora el espacio de soluciones partiendo de la relajación lineal del problema y ramificando progresivamente sobre variables fraccionarias hasta encontrar soluciones enteras válidas, manteniendo en cada paso la mejor solución entera hallada hasta el momento. Cuando el solver alcanza el límite de tiempo antes de probar la optimalidad, devuelve esa mejor solución: factible pero no necesariamente óptima. Esto permite comparar los resultados de MILP en tiempos razonables, al costo de perder la garantía de optimalidad.

3.3.3. Análisis de complejidad del modelo

Para complementar la intuición discutida al inicio de esta sección, se cuantifica a continuación el crecimiento del modelo MILP en función del tamaño de la instancia. Las medidas relevantes son el número de variables y de restricciones generadas, junto con la cantidad de términos no nulos en cada una, ya que estos valores determinan el tamaño de la matriz que el solver debe procesar internamente durante el algoritmo de *branch and bound*.

Se utiliza la siguiente notación: $|V|$ es el número de vehículos, $|C|$ el número de clientes y $|N| = |C| + 2|V|$ el número total de nodos (clientes más nodos de depósito y de retorno por vehículo). El análisis se expresa en notación $\mathcal{O}(\cdot)$: se contabiliza el orden de magnitud de cada familia sin distinguir casos particulares como $i = j$, con el propósito de dar una caracterización de la complejidad del modelo.

Variables. El modelo declara tres familias de variables:

- x_{ij}^k binarias, con $k \in V$, $i, j \in N$: $\mathcal{O}(|V| \cdot |N|^2)$ variables.
- u^k binarias, con $k \in V$: $\mathcal{O}(|V|)$ variables.
- s_i^k reales acotadas por la ventana de tiempo del nodo i (Ec. 2.8), con $k \in V$, $i \in N$: $\mathcal{O}(|V| \cdot |N|)$ variables. Las cotas se aplican como *bounds* de variable y no como restricciones explícitas.

El término dominante corresponde a las variables x_{ij}^k, con un crecimiento $\mathcal{O}(|V| \cdot |N|^2)$.

Restricciones. A continuación se analiza cada familia, contabilizando la cantidad de restricciones generadas y los términos por restricción.

- **Cobertura** (Ec. 2.2): una restricción por cliente $\Rightarrow \mathcal{O}(|C|)$ restricciones, cada una con $\mathcal{O}(|V| \cdot |N|)$ términos. Total: $\mathcal{O}(|C| \cdot |V| \cdot |N|)$.
- **Salida del depósito** (Ec. 2.4): una restricción por vehículo $\Rightarrow \mathcal{O}(|V|)$ restricciones, cada una con $\mathcal{O}(|C|)$ términos. Total: $\mathcal{O}(|V| \cdot |C|)$.
- **Retorno al nodo de retorno** (Ec. 2.6): una restricción por vehículo $\Rightarrow \mathcal{O}(|V|)$ restricciones, cada una con $\mathcal{O}(|N|)$ términos. Total: $\mathcal{O}(|V| \cdot |N|)$.
- **Conservación de flujo** (Ec. 2.5): una restricción por par $(h, k) \in N \times V \Rightarrow \mathcal{O}(|V| \cdot |N|)$ restricciones, cada una con $\mathcal{O}(|N|)$ términos. Total: $\mathcal{O}(|V| \cdot |N|^2)$.
- **Mínimo de vehículos** (Ec. 2.13): una única restricción con $\mathcal{O}(|V|)$ términos. Total: $\mathcal{O}(|V|)$.
- **Capacidad** (Ec. 2.3): una restricción por vehículo $\Rightarrow \mathcal{O}(|V|)$ restricciones, cada una con $\mathcal{O}(|C| \cdot |N|)$ términos. Total: $\mathcal{O}(|V| \cdot |C| \cdot |N|)$.
- **Ventanas de tiempo** (Ec. 2.7, linealizada por Big-M): una restricción por terna $(k, i, j) \in V \times N \times N \Rightarrow \mathcal{O}(|V| \cdot |N|^2)$ restricciones, cada una con $\mathcal{O}(1)$ términos. Total: $\mathcal{O}(|V| \cdot |N|^2)$.
- **Activación de vehículos** (Ec. 2.12): una restricción por vehículo $\Rightarrow \mathcal{O}(|V|)$ restricciones, cada una con $\mathcal{O}(|N|^2)$ términos. Total: $\mathcal{O}(|V| \cdot |N|^2)$.

La Tabla 3.1 resume el aporte de cada familia.

Familia	# restricciones	Términos c/u	Total
Cobertura (Ec. 2.2)	$\mathcal{O}(\|C\|)$	$\mathcal{O}(\|V\| \cdot \|N\|)$	$\mathcal{O}(\|C\| \cdot \|V\| \cdot \|N\|)$
Salida (Ec. 2.4)	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\|C\|)$	$\mathcal{O}(\|V\| \cdot \|C\|)$
Retorno (Ec. 2.6)	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\|N\|)$	$\mathcal{O}(\|V\| \cdot \|N\|)$
Flujo (Ec. 2.5)	$\mathcal{O}(\|V\| \cdot \|N\|)$	$\mathcal{O}(\|N\|)$	$\mathcal{O}(\|V\| \cdot \|N\|^2)$
Mín. vehículos (Ec. 2.13)	$\mathcal{O}(1)$	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\|V\|)$
Capacidad (Ec. 2.3)	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\|C\| \cdot \|N\|)$	$\mathcal{O}(\|V\| \cdot \|C\| \cdot \|N\|)$
Ventanas (Ec. 2.7)	$\mathcal{O}(\|V\| \cdot \|N\|^2)$	$\mathcal{O}(1)$	$\mathcal{O}(\|V\| \cdot \|N\|^2)$
Activación (Ec. 2.12)	$\mathcal{O}(\|V\|)$	$\mathcal{O}(\|N\|^2)$	$\mathcal{O}(\|V\| \cdot \|N\|^2)$

Cuadro 3.1: Crecimiento de cada familia de restricciones del modelo MILP.

Tamaño total del modelo. Los términos dominantes provienen de las restricciones de flujo, ventanas de tiempo y activación, todos con orden $\mathcal{O}(|V| \cdot |N|^2)$. Sumando todas las contribuciones, el tamaño total del modelo crece como:

$$\mathcal{O}\big(|V| \cdot |N|^2\big)$$

Como $|N| = |C| + 2|V|$, esta cota puede reescribirse en función de $|C|$ y $|V|$ como:

$$\mathcal{O}\big(|V| \cdot (|C| + |V|)^2\big) \;=\; \mathcal{O}\big(|V| \cdot |C|^2 + |V|^2 \cdot |C| + |V|^3\big)$$

En el régimen habitual donde $|C| \gg |V|$, el término dominante es $\mathcal{O}(|V| \cdot |C|^2)$; en el caso opuesto, el crecimiento se vuelve cúbico en $|V|$. En ambos extremos, duplicar el tamaño de la instancia implica un crecimiento más que lineal en el tamaño del modelo, lo cual se traduce en un aumento sustancial del tiempo de resolución dado el espacio de búsqueda combinatorio sobre las $\mathcal{O}(|V| \cdot |N|^2)$ variables binarias x_{ij}^k.

Para poner un número concreto, consideremos una instancia con $|C| = 20$ clientes y $|V| = 10$ vehículos. Aplicando la expansión anterior:

$$|V|\cdot|C|^2 + |V|^2\cdot|C| + |V|^3 = 10\cdot 400 + 100\cdot 20 + 1000 = 4000+2000+1000 \approx 7000$$

Es decir, para una instancia de tamaño moderado el modelo ya contiene del orden de varios miles de términos. Si se duplican ambos parámetros a $|C| = 40$ y $|V| = 20$, el mismo cálculo da $20 \cdot 1600 + 400 \cdot 40 + 8000 \approx 56\,000$ términos: ocho veces más para apenas el doble de clientes y vehículos.

Este análisis confirma cuantitativamente la intuición previa: el modelo MILP es de alta dimensionalidad incluso para instancias de tamaño moderado, lo que justifica la necesidad de las heurísticas y metaheurísticas presentadas previamente en este capítulo.

Capítulo 4

Experimentación numérica

Para evaluar empíricamente las estrategias de resolución presentadas en el capítulo anterior, es necesario diseñar un entorno de experimentación que permita compararlas bajo condiciones controladas y reproducibles. Este capítulo describe los pasos que hacen posible esa comparación: la generación de instancias representativas del dominio agrícola, la optimización de los hiperparámetros de ACO, la definición de la configuración experimental y la infraestructura utilizada para garantizar la reproducibilidad de los resultados.

4.1. Generación de instancias

Dado que el problema modelado incorpora características específicas, se optó por generar instancias sintéticas propias en lugar de utilizar *benchmarks* estándar de la literatura como las instancias de Solomon [12]. Esto permite un control preciso sobre las condiciones del experimento y garantiza que las instancias sean consistentes con la variante del problema estudiada.

Se implementaron dos generadores. En ambos casos, la matriz de costos, los tiempos de viaje, las demandas y las capacidades se generan a partir de un tamaño $(|C|, |V|)$, una semilla y un parámetro `cost_range`, que define el rango [low, high] del cual se extraen aleatoriamente los costos entre nodos. Adicionalmente, el generador constructivo recibe el parámetro `tw_tightness`, que define el rango del cual se extrae aleatoriamente el ancho de cada ventana de tiempo: un rango estrecho produce ventanas con poca flexibilidad horaria, mientras que uno amplio da mayor margen al cliente. La diferencia fundamental entre los generadores es el criterio con el que se posicionan esas ventanas.

Generador constructivo. Primero construye una solución de referencia: asigna los clientes a vehículos respetando las restricciones de capacidad y simula el recorrido resultante. Las ventanas de tiempo de cada cliente se centran en su tiempo de llegada real según dicha solución, con un ancho aleatorio controlado por el parámetro `tw_tightness`. El horizonte $T_{\text{máx}}$ se fija con un margen del 50 % sobre la ruta más larga, asegurando factibilidad. Como las ventanas se construyen alrededor de una solución concreta, la factibilidad de la instancia está garantizada por construcción.

Sin embargo, este diseño introduce un sesgo: la solución de referencia se construye de forma greedy por capacidad, de manera análoga a la que emplea TWNN, por lo que las instancias generadas tienden a favorecerlo. La Tabla 4.1 ilustra las ventanas resultantes para un ejemplo de 3 clientes.

`tw_tightness` $= [0,\ 20]$

Cliente	e_i	l_i	**Ancho**
0	11	27	16
1	22	25	3
2	0	13	13

`tw_tightness` $= [0,\ 500]$

Cliente	e_i	l_i	**Ancho**
0	0	49	49
1	0	49	49
2	0	49	49

Cuadro 4.1: Efecto de `tw_tightness` sobre las ventanas generadas ($T_{\text{max}} = 49$, semilla 42, $|C| = 3$, $|V| = 2$). Con `tw_tightness`$= [0, 500]$ las ventanas se expanden hasta cubrir el horizonte completo, eliminando en la práctica las restricciones temporales.

Generador aleatorio. No construye ninguna solución previa. El horizonte $T_{\text{máx}}$ se estima a partir de estadísticas de la matriz de tiempos de viaje, y las ventanas de tiempo se ubican aleatoriamente dentro de $[0,\ T_{\text{máx}}]$, verificando únicamente que cada cliente sea alcanzable desde algún depósito y pueda retornar en tiempo. Como resultado, las instancias generadas pueden ser infactibles; el criterio de inclusión en el análisis se detalla en la Sección 5.2. La Tabla 4.2 muestra las ventanas para el mismo tamaño de instancia.

Cliente	e_i	l_i	**Ancho**
0	0	25	25
1	14	30	16
2	3	28	25

Cuadro 4.2: Ventanas de tiempo generadas por el generador aleatorio ($T_{\text{max}} = 32$, semilla 42, $|C| = 3$, $|V| = 2$).

4.2. Optimización de hiperparámetros

El rendimiento de ACO depende fuertemente de sus hiperparámetros: los coeficientes α, β, γ, la tasa de evaporación ρ, el peso de urgencia W_u, la probabilidad de explotación q_0, entre otros. Elegir estos valores manualmente es difícil porque interactúan entre sí de formas no lineales y su efecto varía según las características de las instancias.

La optimización de hiperparámetros puede formularse como un problema de optimización en sí mismo: dado un algoritmo parametrizado, encontrar la configuración que minimiza una métrica de rendimiento sobre un conjunto representativo de instancias. La dificultad radica en que cada evaluación requiere ejecutar el algoritmo completo, lo que hace prohibitivo explorar el espacio de configuraciones de forma exhaustiva. Estrategias simples como la búsqueda en grilla o la búsqueda aleatoria no escalan bien cuando el número de parámetros es alto y sus interacciones entre sí son no lineales: el valor óptimo de un parámetro depende del valor de los demás, por lo que no es posible optimizarlos de forma independiente. La optimización bayesiana aborda este problema construyendo un modelo probabilístico del espacio de búsqueda que se actualiza con cada evaluación, permitiendo proponer configuraciones prometedoras en lugar de explorar a ciegas.

Por eso se utilizó Optuna [13], un *framework* de optimización automática de hiperparámetros basado en el algoritmo TPE (*Tree-structured Parzen Estimator*), que construye un modelo probabilístico del espacio de búsqueda y lo usa para proponer configuraciones cada vez más prometedoras en lugar de explorar a ciegas.

El objetivo de la optimización fue minimizar el costo promedio de las soluciones encontradas por ACO sobre un conjunto de instancias representativo. La configuración resultante no solo redujo el costo promedio, sino que también disminuyó significativamente la dispersión entre ejecuciones, lo que permitió correr el algoritmo una sola vez por instancia en lugar de múltiples repeticiones a pesar de su naturaleza estocástica. Las instancias utilizadas para la calibración fueron generadas con el mismo rango de parámetros que el experimento principal (véase la Sección 4.3), pero con distintas combinaciones, evitando así que el algoritmo se calibre sobre las mismas instancias que luego se evalúan.

4.3. Configuración experimental

Para reducir la combinatoria experimental, se fijaron los parámetros que no son objeto de análisis comparativo, variando únicamente el tamaño de la instancia.

La Tabla 4.3 resume los parámetros fijos del experimento.

Parámetro	**Valor**	**Descripción**
`customer_demand`	1	Demanda uniforme por cliente
`capacity`	100	Capacidad uniforme por vehículo
`vehicle_penalty`	0	Sin penalización por uso de vehículo
`cost_range`	$[1, 20]$	Rango de costos de la matriz
`tw_tightness`	$[0, 500]$	Amplitud de ventanas de tiempo
`seed`	42	Semilla base para reproducibilidad
`instance_reps`	1	Repeticiones por instancia (ACO es estocástico)
`aco_duration`	80 s	Duración de ejecución (ACO)
`timeout`	1200 s	Tiempo límite por ejecución (MILP)
`memory_limit`	10 GB	Límite de RAM por ejecución

Cuadro 4.3: Parámetros fijos del experimento.

La Tabla 4.4 presenta los parámetros de ACO, determinados mediante la optimización descripta en la Sección 4.2.

Parámetro	**Valor**	**Descripción**
α	0.785	Influencia de las feromonas
β	0.774	Influencia de la heurística
ρ	0.185	Tasa de evaporación
λ	5	Penalización por clientes no atendidos
γ	2	Exponente del ahorro de Clarke-Wright
Φ	97	Constante de escala del refuerzo
$\tau_{\text{máx}}$	15	Cota superior de feromonas
$\tau_{\text{mín}}$	0.002	Cota inferior de feromonas
W_u	1.473	Peso de urgencia en el criterio de ruteo
q_0	0.815	Probabilidad de explotación ACS
`num_ants`	28	Número de hormigas por iteración

Cuadro 4.4: Parámetros fijos de ACO.

Se variaron el número de clientes $|C|$ y el número de vehículos $|V|$ para evaluar el comportamiento de cada algoritmo en distintos regímenes de escala. Los valores considerados para $|C|$ fueron $\{5,\ 10,\ 20,\ 40,\ 60,\ 80\}$. Para cada valor de $|C|$ se evaluaron tres relaciones de flota: $|V| = \lfloor |C|/2 \rfloor$, $|V| = |C|$ y $|V| = 2 \cdot |C|$, representando escenarios con flota escasa, balanceada y abundante respectivamente.

Cada instancia fue evaluada con los cuatro solvers implementados: TWNN, ACO, Gurobi y HiGHS. El experimento produjo un total de 30 instancias y 120 ejecuciones.

4.4. Infraestructura y reproducibilidad

La configuración de los experimentos se gestionó mediante Hydra [14], un *framework* que permite organizar y sobreescribir parámetros de forma declarativa sin modificar el código. Cada combinación de parámetros se define en archivos `YAML` y puede ser lanzada como un experimento independiente.

Cada ejecución de MILP estuvo sujeta a un límite de tiempo de 1200 segundos. Al alcanzar este límite, la ejecución se registra con estado `timeout`; si el solver había encontrado una solución factible antes del corte, ésta queda registrada aunque no haya sido probada como óptima. Adicionalmente, cada proceso estuvo limitado a 10 GB de RAM; si superaba ese umbral, era interrumpido abruptamente y registrado con estado `OOM`, sin retornar ninguna solución.

Por cada ejecución se registraron: costo de la solución, factibilidad, validez, tiempo de cómputo, rutas obtenidas, clientes no atendidos y estado de terminación (`finished`, `timeout` o `OOM`).

Los experimentos fueron ejecutados en un equipo con procesador Intel Core i7-10700F (2,90 GHz base, 4,80 GHz máx., 8 núcleos / 16 hilos), 16 GB de RAM y sistema operativo Ubuntu 24.04.4 LTS (x86_64).

Capítulo 5

Resultados

5.1. Criterios de evaluación

Antes de presentar los resultados, se definen los criterios con los que se evalúan las ejecuciones a lo largo del capítulo.

Una solución se considera **factible** si satisface todas las restricciones del problema: cobertura de clientes, capacidad vehicular, ventanas de tiempo y flujo de rutas.

Cada ejecución puede finalizar en uno de tres estados: **finished** (el solver completó la ejecución dentro del tiempo asignado), **timeout** (se agotó el tiempo límite, pudiendo o no haber encontrado una solución factible) u **OOM** (*out of memory*, el proceso fue interrumpido abruptamente por agotamiento de la memoria RAM disponible, sin retornar ningún resultado).

A lo largo de este capítulo, el tamaño del problema se mide en $|C|$ o en $|N|$, conforme a la notación introducida en la Sección 3.3.3. En los gráficos donde el mensaje central refiere a un umbral en la cantidad de clientes se utiliza directamente $|C|$. En aquellos donde se grafican simultáneamente todas las combinaciones $(|C|, |V|)$, se emplea $|N|$, que evita superposiciones en el eje horizontal cuando distintas combinaciones comparten el mismo valor de $|C|$.

5.2. Comparación

El conjunto experimental consta de 30 instancias: 15 generadas mediante el generador constructivo y 15 mediante el generador aleatorio, ambos descriptos en la Sección 4.1. Con cuatro metodologías evaluadas (TWNN, ACO, Gurobi y HiGHS), el experimento produce un total de 120 ejecuciones.

Como se discutió en la Sección 4.1, las instancias del generador aleatorio pueden ser infactibles. Para garantizar la validez de la comparación se establece el siguiente criterio: una instancia se descarta si ningún solver encontró una solución factible para ella. Si al menos uno la resuelve, la instancia es factible y los fallos de los restantes se interpretan como incapacidad del algoritmo o limitación de recursos, según el caso, pero no como evidencia de infactibilidad. En la práctica, este caso no se presentó en ninguna de las 15 instancias del generador aleatorio.

La Figura 5.1 presenta la tasa de cobertura de cada metodología, es decir, la proporción de instancias para las cuales se obtuvo una solución factible, en función del número de clientes. A primera vista se aprecia una jerarquía clara: ACO y Gurobi exhiben la mayor cobertura, manteniéndose en el 100 % durante la mayor parte del rango evaluado; HiGHS presenta un deterioro más temprano, con caídas marcadas a partir de $|C| = 35$. TWNN muestra un patrón diferente y más irregular. Como se discutió en la Sección 4.1, el generador constructivo produce venta-

nas de tiempo centradas en una solución de referencia análoga a la que TWNN construiría, lo que lo favorece directamente: sobre las 15 instancias constructivas obtuvo solución factible en los 15 casos. En cambio, sobre las 15 instancias del generador aleatorio solo logró resolver 3, explicando la mayor parte de la irregularidad observada en el gráfico. Dentro de las instancias que sí resuelve, la cobertura es mayor cuando la flota es balanceada ($|V| = |C|$), seguida por el caso de flota abundante ($|V| = 2|C|$): a mayor disponibilidad de vehículos relativa a clientes, la asignación greedy cuenta con más margen para respetar las ventanas de tiempo.

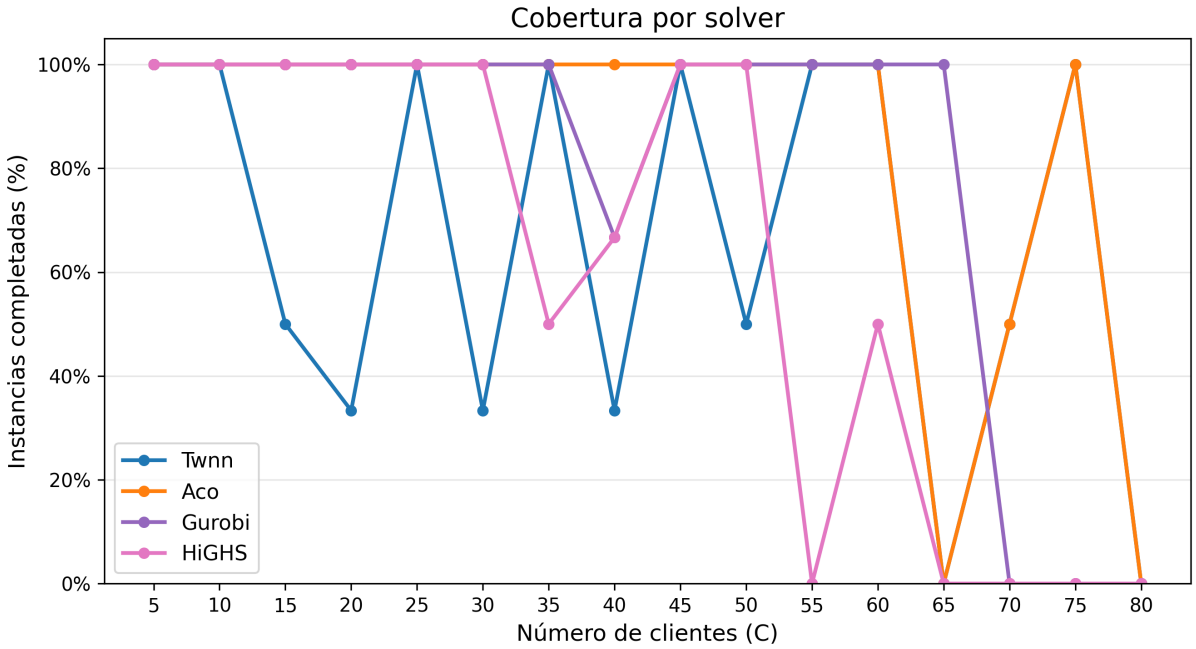


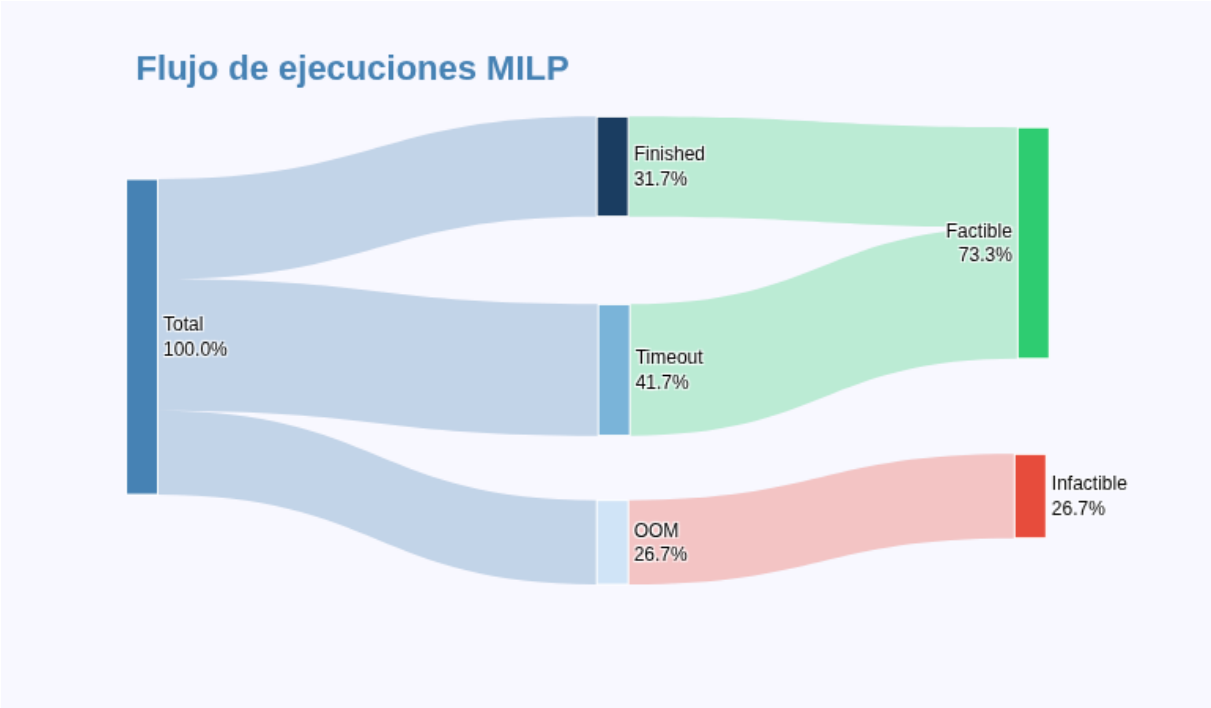
Figura 5.1: Cobertura por solver en función del número de clientes.

Volviendo a la Figura 5.1, el patrón observado plantea dos preguntas:

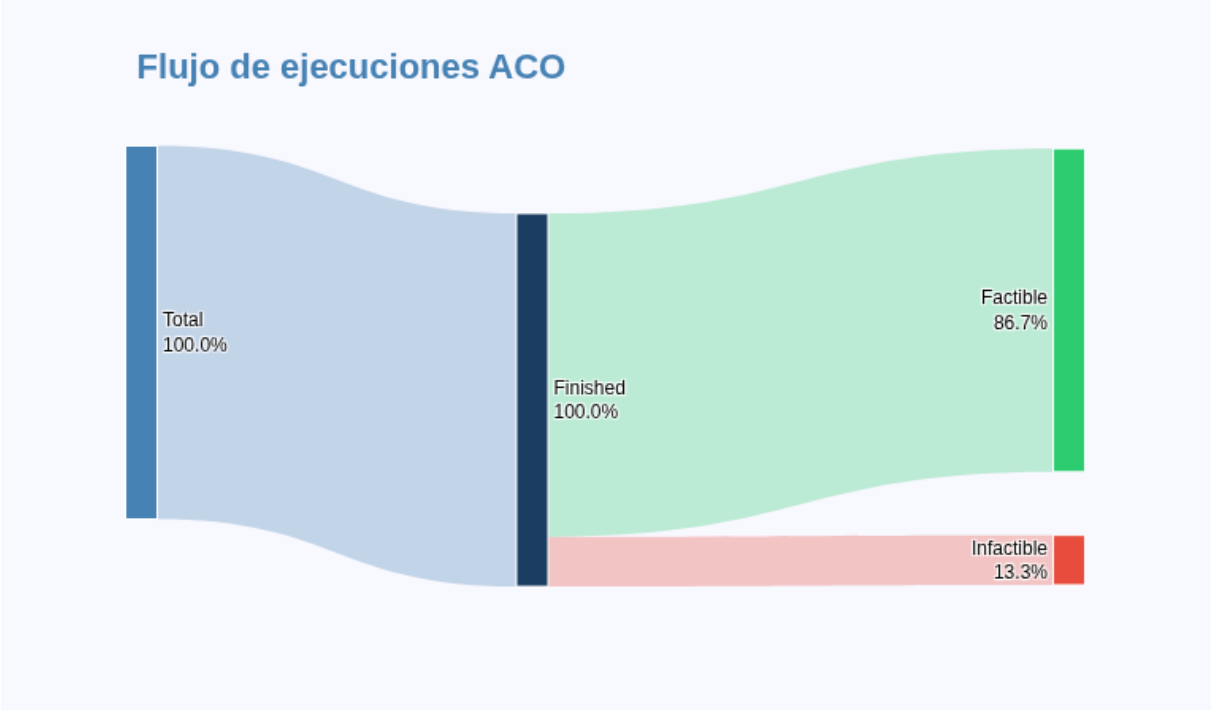
1. ¿A qué se deben las pérdidas de cobertura y cuáles son sus causas en cada caso?
2. Cuando los solvers sí encuentran solución, ¿qué tan buena es esa solución?

Las Secciones 5.2.1 y 5.2.2 abordan estas preguntas respectivamente.

5.2.1. Causas de las pérdidas de cobertura



(a) Flujo de ejecuciones MILP.



(b) Flujo de ejecuciones ACO.

Figura 5.2: Comparación del flujo de ejecuciones entre MILP y ACO.

El contraste es marcado. ACO completa el 100 % de sus ejecuciones sin restricciones de recursos y obtiene soluciones factibles en el 86.7 % de los casos; el 13.3 % restante corresponde

a instancias en las que el algoritmo no logró construir una solución válida, sin mediar limitaciones de memoria ni de tiempo. MILP, en cambio, solo completa exitosamente el 31.7 % de sus ejecuciones: el 41.7 % agota el tiempo límite y el 26.7 % restante es interrumpido por falta de memoria (OOM). Las ejecuciones con OOM son interrumpidas por el framework sin retornar ningún resultado; las que alcanzan el timeout, en cambio, retornan la mejor solución factible encontrada hasta el momento, en caso de contar con un incumbente. En conjunto, el 73.4 % de las ejecuciones MILP produce alguna solución factible (aunque sin garantía de optimalidad en los casos de timeout), mientras que el 26.6 % restante no retorna resultado utilizable. Las subsecciones siguientes examinan en detalle el impacto del crecimiento del modelo sobre el uso de memoria, y el crecimiento del tiempo de resolución con el tamaño de la instancia.

Límite de memoria (OOM)

La Figura 5.3 presenta el uso de RAM por metodología. TWNN y ACO no requieren cantidades significativas de memoria: su representación del problema no crece de forma estructural con el tamaño de la instancia. MILP, en cambio, construye internamente una representación matricial cuyo tamaño crece con $|V| \cdot |N|^2$, como se analizó en la Sección 3.3.3, lo que se traduce en un consumo que escala rápidamente con el tamaño de la instancia.

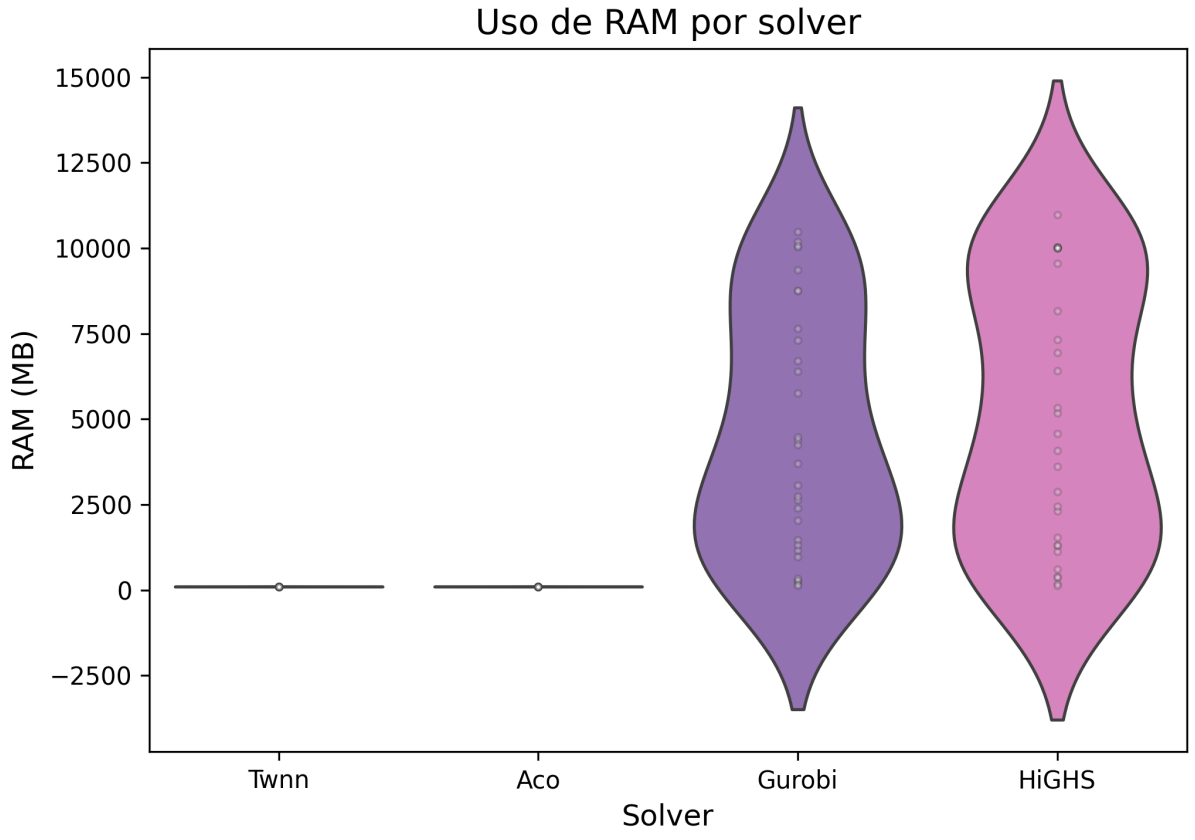


Figura 5.3: Uso de RAM por metodología.

La Figura 5.4 ilustra cómo evoluciona ese consumo a medida que crece el tamaño de las instancias. Del total de casos terminados por OOM, el 62.5 % corresponde a HiGHS y el 37.5 % restante a Gurobi, lo que sugiere que HiGHS presenta una gestión de memoria menos eficiente frente a instancias de mayor dimensión.

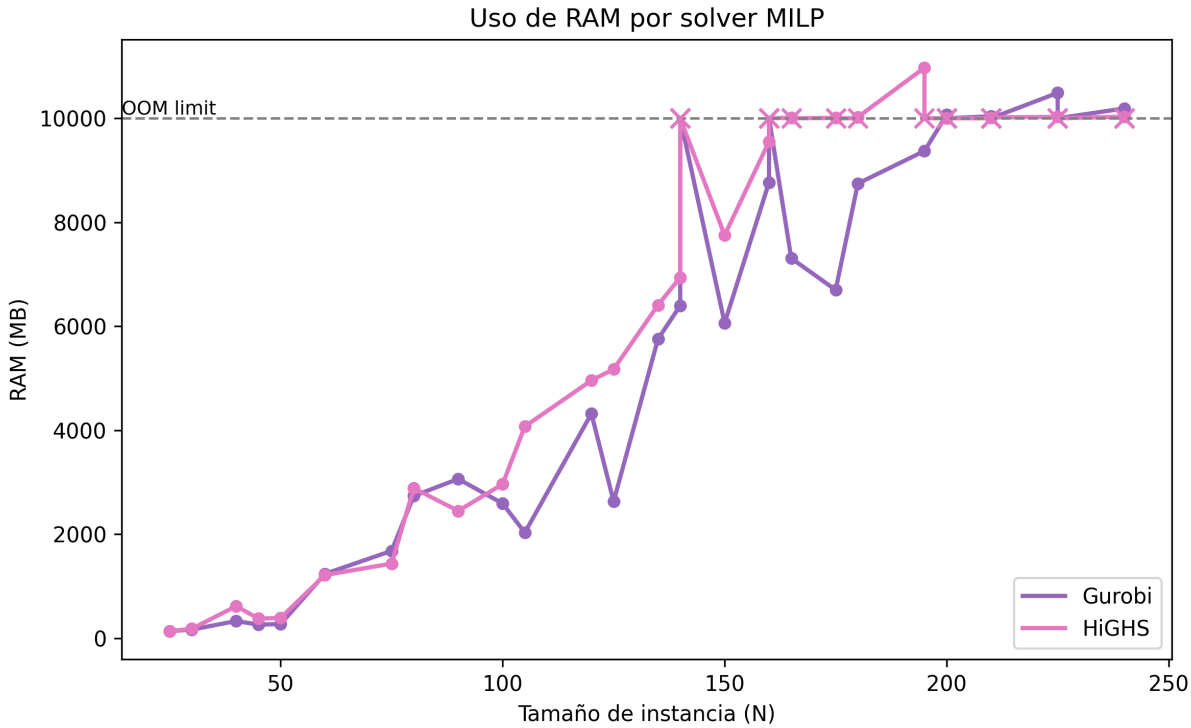


Figura 5.4: Límite de RAM alcanzado.

Aprovechando que se observó el uso de RAM, la Figura 5.5 permite comparar el uso de CPU entre metodologías. TWNN y ACO operan sobre un único núcleo; ninguno fue implementado con soporte de paralelización. Entre los solvers MILP, HiGHS también opera en un único núcleo, mientras que Gurobi distribuye el cómputo entre múltiples núcleos en paralelo, lo que es presumiblemente uno de los factores que explica su ventaja en tiempos de resolución.

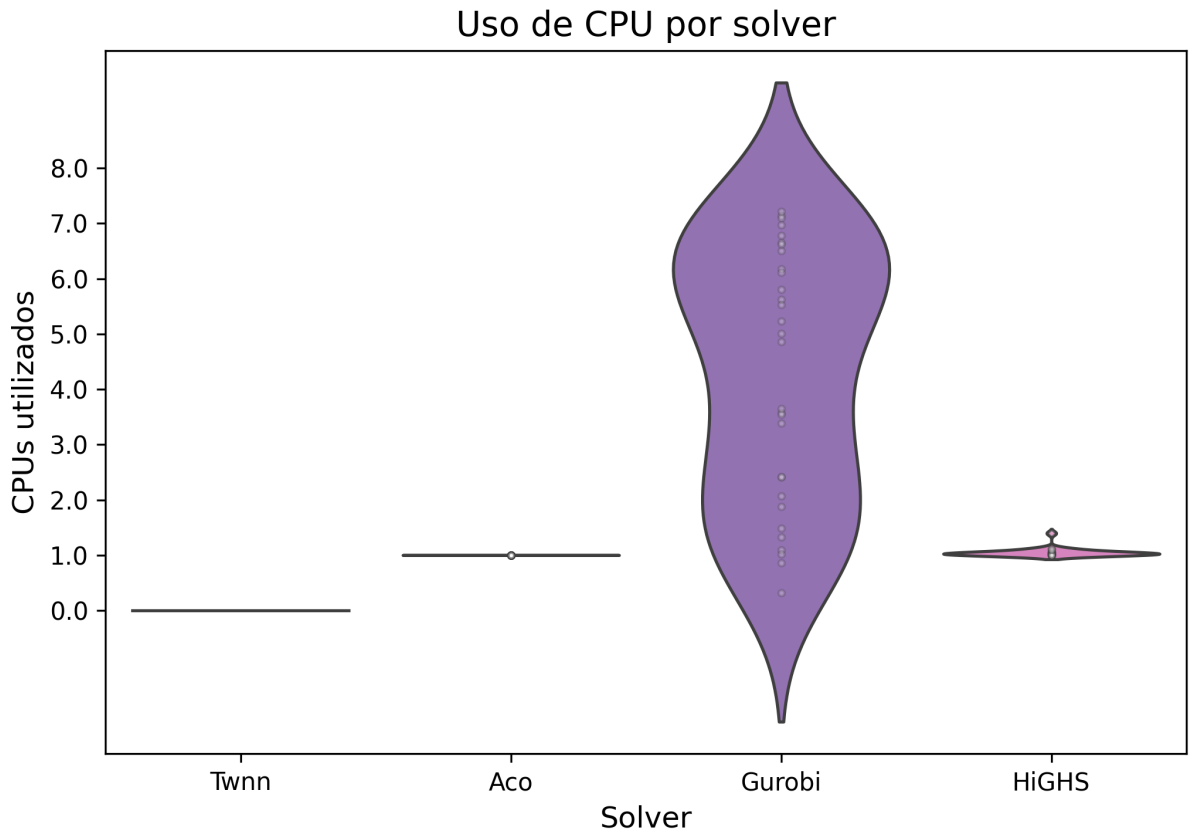


Figura 5.5: Uso de CPU por metodología.

Agotamiento del tiempo límite

La Figura 5.6 presenta el tiempo de cómputo de cada solver MILP en función del tamaño de la instancia, en escala logarítmica. Se distinguen tres fases bien diferenciadas.

Para instancias pequeñas ($|N|50$), los runtimes son del orden de segundos y ambos solvers completan la ejecución sin inconvenientes. A medida que el tamaño crece, el tiempo de resolución escala de forma superlineal y ambos solvers comienzan a alcanzar el límite de 1200 segundos: a partir de aproximadamente $|N| = 60$, la mayoría de las ejecuciones se concentran sobre la línea de timeout. Esta segunda fase se extiende hasta aproximadamente $|N| \approx 160$, donde se observa una caída de los tiempos registrados por debajo del límite.

Esta caída no indica que los solvers se vuelvan más eficientes para instancias grandes: refleja que a ese tamaño el proceso es terminado por OOM antes de alcanzar los 1200 segundos. A partir de $|N| \approx 160$, el OOM desplaza al timeout como mecanismo de falla dominante, y los tiempos registrados corresponden al momento en que el framework interrumpe la ejecución por falta de memoria.

En total, el 41.7 % de las ejecuciones MILP terminan por timeout, de las cuales el 56 % corresponde a HiGHS y el 44 % a Gurobi.

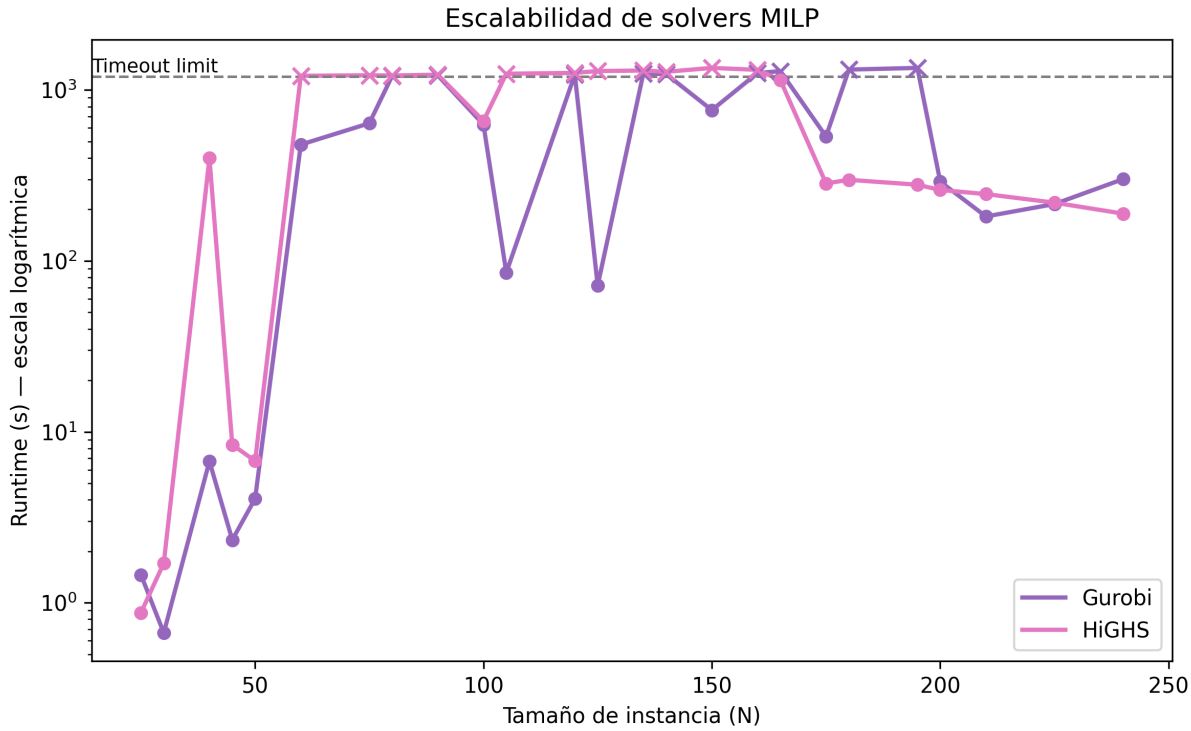


Figura 5.6: Escalabilidad del tiempo de cómputo por solver.

Comparación entre solvers MILP: Gurobi vs. HiGHS

Como se observó en las secciones anteriores, Gurobi supera a HiGHS tanto en la proporción de casos terminados por OOM (37.5 % vs. 62.5 %) como en la de casos terminados por timeout (44 % vs. 56 %). Esta ventaja acumulada se refleja directamente en la cobertura: Gurobi resolvió 24 de las 30 instancias del experimento, mientras que HiGHS resolvió 20.

La Figura 5.7 presenta el perfil de rendimiento de ambos solvers: la fracción acumulada de instancias resueltas en función del tiempo de cómputo. Gurobi se mantiene por delante de HiGHS a lo largo de todo el gráfico, lo que indica que su ventaja no se limita a las instancias más grandes sino que es consistente en todo el rango de tiempos.

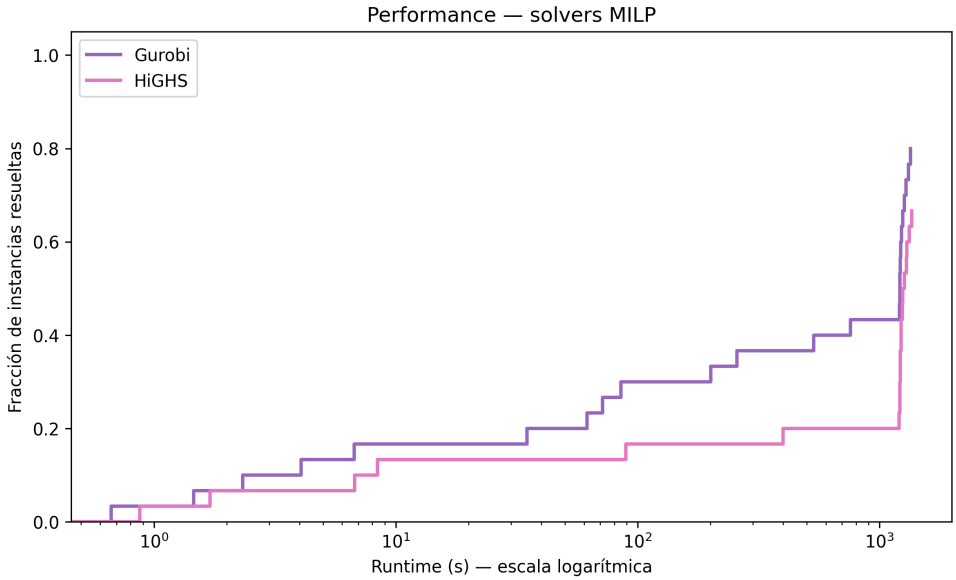


Figura 5.7: Perfil de rendimiento: fracción de instancias resueltas en función del tiempo de cómputo acumulado.

La diferencia puede atribuirse a un conjunto de factores. Entre los algorítmicos, Gurobi invierte fuertemente en preprocesamiento multi-fila para reducir y fortalecer el modelo antes de entrar al branch-and-bound [23]. HiGHS, por su parte, optimiza el núcleo numérico del solver mediante factorización eficiente del sistema de Newton y subrutinas densas [24]. A nivel de recursos, Gurobi distribuye el cómputo entre los 8 núcleos disponibles del equipo, mientras que HiGHS opera en un único núcleo. Sin embargo, hasta donde se relevó, no se encontraron estudios que comparen explícitamente las diferencias internas entre ambos solvers en el contexto del VRP. Existen comparaciones de rendimiento entre múltiples solvers MILP [25], aunque sin aislar las causas de las diferencias observadas.

5.2.2. Calidad de las soluciones

Cubierto el análisis de cobertura, la pregunta siguiente refiere a la calidad de las soluciones encontradas. A continuación se comparan los costos obtenidos por cada solver. Para que la comparación sea justa, cada análisis se restringe al conjunto de instancias donde todos los solvers incluidos en la figura obtuvieron solución factible.

Distribución de costos

La Figura 5.8 presenta la distribución de costos sobre el conjunto de 18 instancias en que los cuatro solvers (TWNN, ACO, HiGHS y Gurobi) encontraron solución factible. Se distinguen cuatro niveles: Gurobi obtiene los menores costos (media 109.4), seguido por HiGHS (141.1) y ACO (163.7). TWNN se ubica en una categoría diferente, con una media de 662.8 y una dispersión considerablemente mayor que el resto.

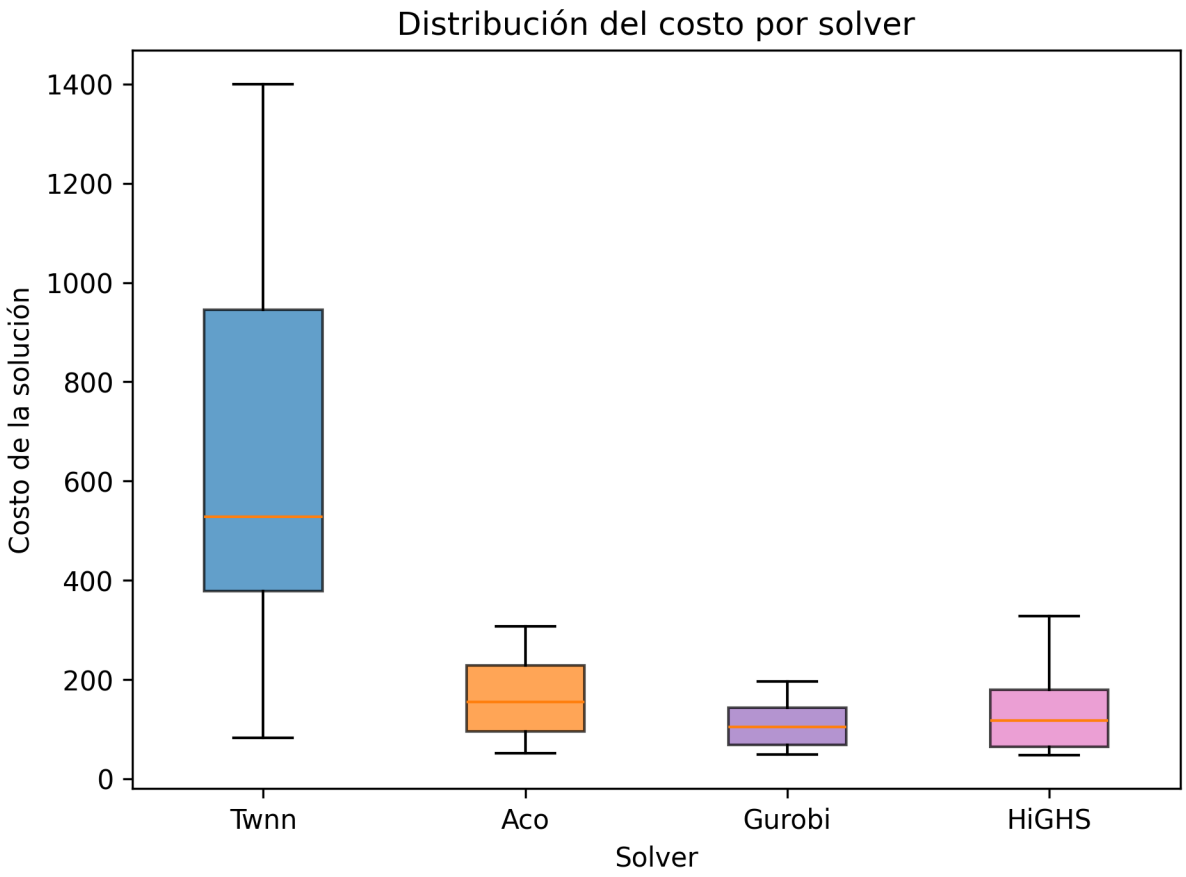


Figura 5.8: Distribución de costos de solución por solver.

Gap respecto a Gurobi

La Figura 5.9 presenta el gap relativo de HiGHS y ACO respecto a Gurobi. TWNN se excluye de esta figura por tener gaps de un orden de magnitud superior que distorsionarían la escala. Al no incluir TWNN, el factor limitante pasa a ser HiGHS, que resolvió 20 de las 30 instancias; el análisis se realiza sobre ese subconjunto. HiGHS presenta un gap medio de 20.8 % con alta dispersión: su rango va de -2.0 % a 69.2 %, lo que refleja un comportamiento inconsistente entre instancias. ACO presenta un gap medio de 33.1 % con un rango similar (0 % a 68.5 %) pero mayor concentración alrededor de la media. Los estadísticos completos se detallan en la Tabla 5.1.

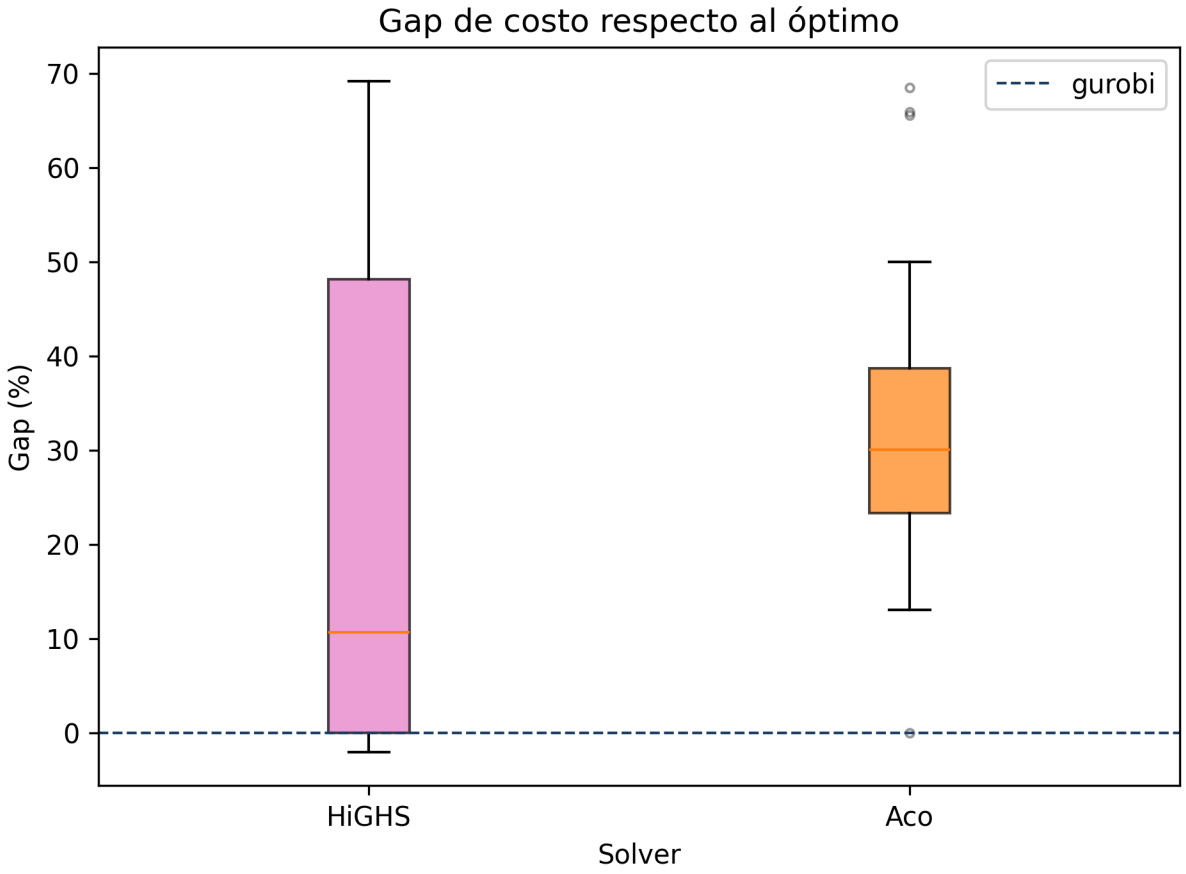


Figura 5.9: Gap relativo al óptimo de Gurobi por solver.

El gap negativo de HiGHS (-2.0 %) indica que en alguna instancia HiGHS encontró una solución de menor costo que Gurobi: ambos solvers alcanzaron el timeout retornando una solución subóptima, pero la de HiGHS resultó ser de menor costo. ACO, en cambio, no supera a Gurobi en ninguna instancia.

La diferencia de gap entre ACO y HiGHS debe leerse en contexto. Por un lado, ACO opera con un límite de 80 segundos mientras que MILP dispone de hasta 1200, por lo que parte del gap es atribuible a esta diferencia de recursos computacionales. Por otro lado, como se discutió en la Sección 3.2.2, la búsqueda local de ACO opera únicamente a nivel intra-ruta, sin intercambio de clientes entre rutas distintas, lo que deja margen de mejora estructural. Como línea de trabajo futuro, paralelizar la construcción de soluciones entre hormigas (actualmente secuencial) permitiría explorar un mayor número de iteraciones en el mismo tiempo de cómputo, con potencial de reducir este gap sin aumentar el límite de tiempo.

Cuadro 5.1: Estadísticos del gap respecto a Gurobi.

Solver	Mínimo	Máximo	Media	Desvío estándar
HiGHS	-2.0 %	69.2 %	20.8 %	26.5 %
ACO	0.0 %	68.5 %	33.1 %	18.2 %

La Figura 5.10 desagrega el gap promedio por configuración (c, v). El patrón disperso del heatmap refleja el diseño experimental: en lugar de evaluar todas las combinaciones posibles de c y v, se acotó el espacio a tres relaciones entre ambos ($c = v$, $c = 2v$ y $v = 2c$) para mantener el

experimento dentro de un alcance razonable. El gap no es uniforme: instancias con pocos clientes ($c = 5$) resultan triviales para ACO (0 %), mientras que configuraciones de tamaño moderado presentan los gaps más elevados. La combinación con mayor gap promedio es $c = 50$, $v = 25$ con un 68.5 %, que se toma como caso de estudio en la figura siguiente.

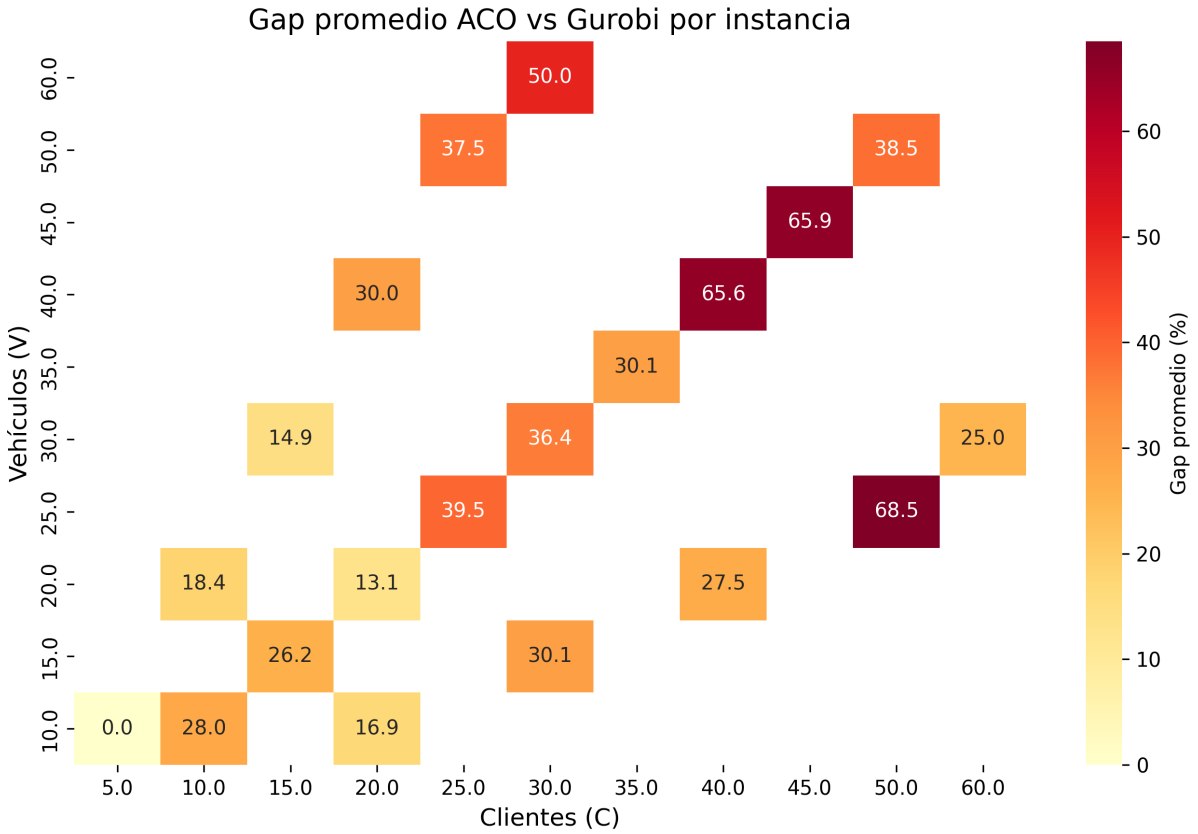


Figura 5.10: Gap promedio de ACO respecto a Gurobi por configuración (c, v).

La Figura 5.11 muestra la evolución del costo de ACO sobre esa instancia, ejecutada hasta los 600 segundos (la mitad del tiempo disponible para MILP). La línea roja indica el corte utilizado en el experimento (80 s); la línea verde, los 300 segundos.

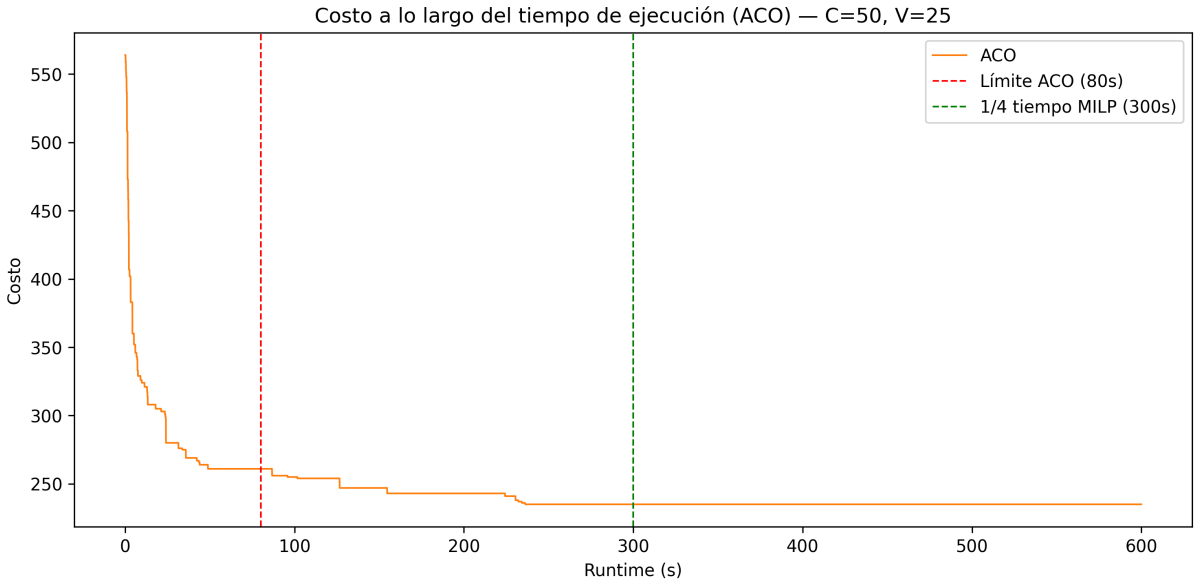


Figura 5.11: Evolución del costo de ACO en la instancia de mayor gap ($c = 50$, $v = 25$).

La curva muestra que al cortar en 80 segundos, ACO no ha convergido: la solución continúa mejorando hasta aproximadamente los 300 segundos, acumulando una reducción de costo del 9.96 %. A partir de ese punto la curva se estabiliza, lo que indica que la mejora restante ya no depende de exploración sino de limitaciones estructurales del algoritmo (ausencia de búsqueda local inter-ruta). Cabe señalar que, dado que ACO es estocástico y en el experimento principal se ejecutó una sola vez por instancia, el gap puntual al segundo 80 varía entre corridas; este análisis de convergencia se realizó sobre una ejecución extendida durante la etapa de análisis, precisamente para entender si el límite temporal era el factor limitante. El límite de 80 segundos se estableció siguiendo la práctica habitual de la literatura, que típicamente utiliza 60 segundos, ampliando ese margen para dar mayor oportunidad al algoritmo. Correr hasta la meseta (300 s) no se contempló como configuración del experimento principal porque ese umbral se identificó recién en esta etapa de análisis posterior. Aun así, el potencial de mejora dentro del mismo límite temporal es concreto: paralelizar la construcción de soluciones entre hormigas permitiría explorar más iteraciones en esos mismos 80 segundos, logrando un efecto equivalente a extender el tiempo de ejecución de forma secuencial.

Origen de la diferencia en los costos

La Figura 5.12 presenta la fracción de vehículos utilizados sobre el total disponible para cada solver. ACO y Gurobi presentan proporciones comparables a lo largo de todo el rango, mientras que TWNN utiliza sistemáticamente una fracción mayor de la flota disponible. Esto no es una elección del algoritmo sino una limitación estructural: sin ningún mecanismo de reasignación, TWNN no es capaz de reducir la cantidad de vehículos activos una vez que los clientes han sido asignados de forma greedy.

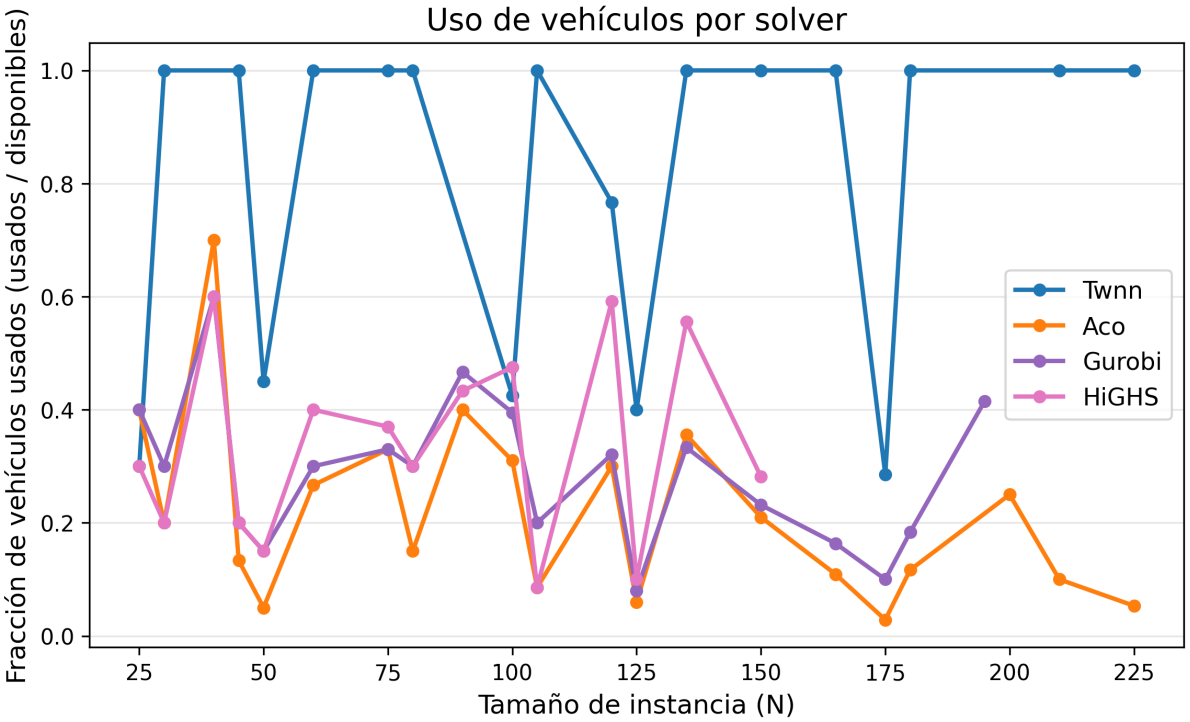


Figura 5.12: Fracción de vehículos utilizados por solver.

La Figura 5.13 ilustra esta diferencia en una instancia representativa ($|C| = 10$, $|V| = 10$). TWNN activa los 10 vehículos disponibles y obtiene un costo de 187; ACO, Gurobi y HiGHS utilizan únicamente 4. ACO coincide exactamente con Gurobi y HiGHS en la cantidad de vehículos empleados. La diferencia de costo entre ACO (76) y Gurobi (69) proviene de dos factores: la asignación de clientes a vehículos y el orden en que se atienden dentro de cada ruta. ACO ya identifica correctamente cuántos vehículos activar; lo que resta es explorar más el espacio de búsqueda para mejorar tanto la distribución de clientes entre vehículos como el orden de visita dentro de cada ruta.

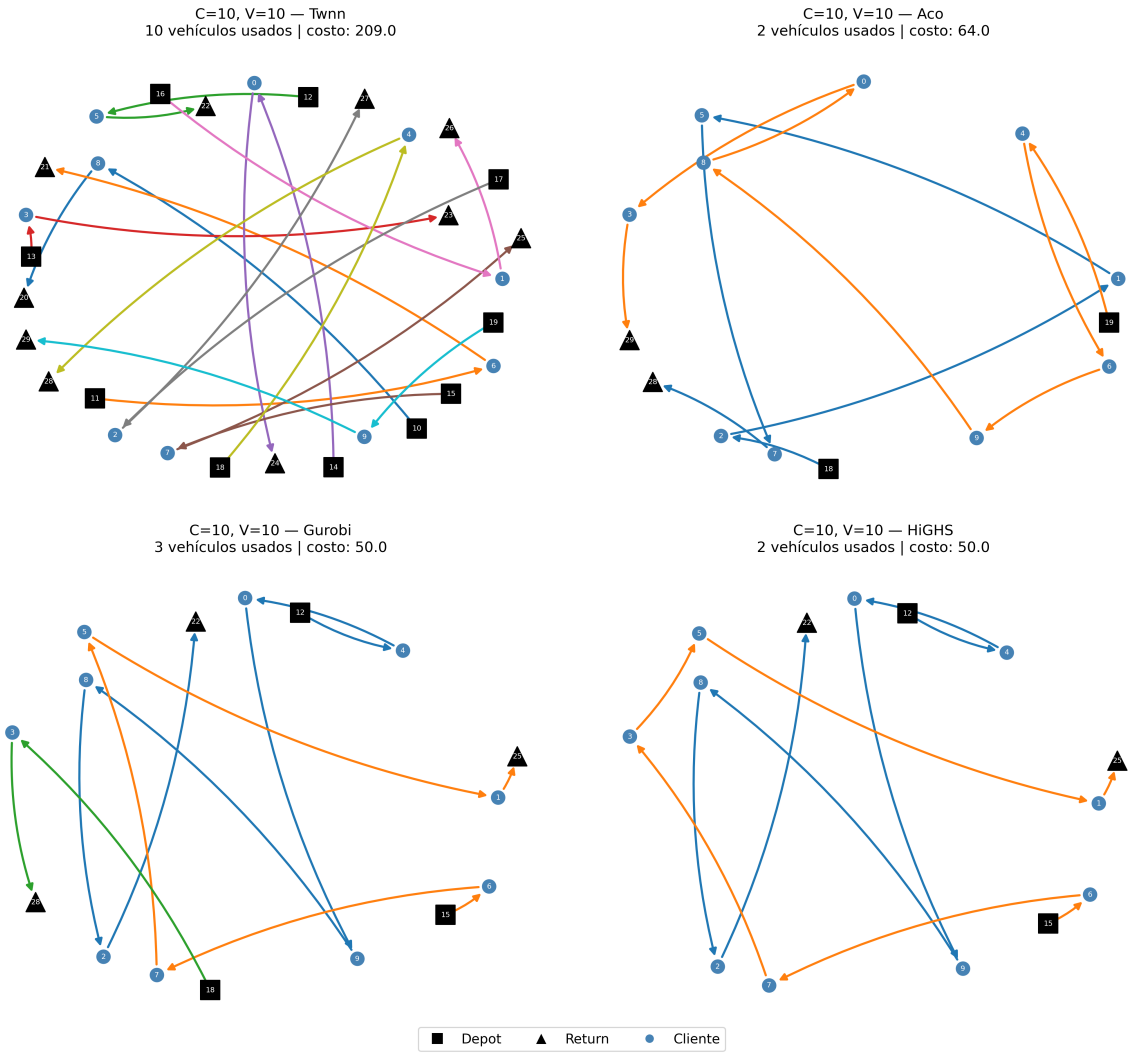


Figura 5.13: Visualización de asignaciones de clientes a rutas.

Síntesis

Ninguna estrategia domina en todas las dimensiones relevantes. MILP garantiza optimalidad pero su aplicabilidad está acotada por los recursos: OOM y timeout limitan su cobertura incluso en instancias de tamaño moderado. Entre ambos solvers exactos, Gurobi demostró un rendimiento superior a HiGHS tanto en instancias resueltas (24 vs. 20 de 30) como en tiempo de cómputo. Todas las ejecuciones de TWNN terminan en menos de 0.01 segundos; sin embargo, el 40 % resultan infactibles, y las que sí encuentran solución lo hacen a un costo medio de 662.8 frente al 109.4 de Gurobi.

ACO representa el balance más favorable: resolvió 26 de las 30 instancias con un gap medio del 33.1 % respecto al óptimo de Gurobi, operando en tan solo 80 segundos frente a los 1200 de MILP. Este resultado cobra mayor valor al considerar que corresponde a una versión base del algoritmo. Como se observó en el análisis de asignaciones, ACO ya logra identificar la cantidad correcta de vehículos a activar; la brecha remanente se concentra en la asignación de clientes a vehículos y el orden de visita dentro de cada ruta, dos aspectos sobre los cuales las mejoras identificadas actúan directamente. La paralelización de la construcción de soluciones entre hormigas y la incorporación de búsqueda local inter-ruta tienen potencial concreto para reducir ese gap de forma significativa, posicionando a ACO como una herramienta prometedora

para un problema con aplicación directa en la logística agrícola cotidiana.

Capítulo 6

Conclusiones

El trabajo comenzó con una revisión de la literatura de VRP para identificar qué variantes habían sido estudiadas, cuáles eran aplicables al dominio agrícola y qué enfoques de resolución mostraban mejores resultados en problemas similares. A partir de esa investigación se identificó la variante más adecuada para representar las restricciones del transporte de maquinaria agrícola en Argentina: múltiples depósitos, ventanas de tiempo, flota heterogénea, capacidad vehicular y tiempos de servicio. Con esa caracterización se formuló un modelo matemático que captura estas restricciones de forma explícita y rigurosa.

Con el modelo definido, se implementaron tres estrategias de resolución de complejidad creciente. TWNN establece una línea de base con una heurística constructiva *greedy*; ACO incorpora mecanismos de exploración y memoria colectiva que permiten escapar de soluciones localmente subóptimas; MILP provee una garantía formal de optimalidad al costo de mayor demanda computacional.

Para hacer comparable esa evaluación, se desarrolló un framework de simulación que automatiza la generación de instancias, la ejecución de los solvers, el monitoreo de recursos y el registro de resultados, garantizando la reproducibilidad del experimento.

Los resultados ubican a ACO como la estrategia con mayor potencial para escalar a instancias reales: logra un balance entre calidad de solución y tiempo de cómputo que ninguna de las otras estrategias alcanza, y el análisis muestra que la brecha con el óptimo es atacable mediante mejoras concretas e identificadas. El trabajo deja un punto de partida sólido para continuar explorando el problema con herramientas más sofisticadas.

Desde una perspectiva práctica, la elección entre ACO y MILP depende del contexto operativo. Si se requiere una solución aceptable en un período corto de tiempo (por ejemplo, para generar una cotización en el momento o reprogramar rutas ante un imprevisto), ACO representa una alternativa válida: produce soluciones en aproximadamente 80 segundos con un gap promedio de 33 % respecto al óptimo. Si, en cambio, el tiempo de cómputo y los recursos computacionales no son una limitante, Gurobi permite alcanzar o aproximar el óptimo de forma más precisa. Esta distinción es relevante en el dominio agrícola, donde las condiciones operativas pueden variar considerablemente entre quienes necesitan una respuesta inmediata y quienes pueden planificar con anticipación.

6.1. Líneas de trabajo futuro

A lo largo del documento se fueron identificando extensiones y mejoras con el fin de incrementar el rendimiento de las distintas metodologías. Esta sección las reúne.

1. **Extensión al modelo general de pickup and delivery.** El modelo actual adopta supuestos simplificadores sobre la estructura de los viajes: la entrega es secuencial, no existen ventanas de tiempo en el destino, y no se permite transportar pedidos de múltiples clientes de forma simultánea. Levantar estos supuestos y extender la formulación al caso general del PDP, incorporando restricciones explícitas de precedencia y acoplamiento entre puntos de origen y destino, ampliaría el rango de escenarios modelables con mayor fidelidad al dominio real.

2. **Precio por kilómetro diferenciado por transportista.** El modelo calcula el costo de cada viaje utilizando una distancia uniforme d_{ij}. En la práctica, cada transportista puede tener una tarifa propia, lo que hace conveniente reemplazar d_{ij} por $d_{ij} \cdot p_k$, donde p_k es el precio por kilómetro del vehículo k. Esta extensión es directamente compatible con la formulación actual y permitiría reflejar estructuras de costos más realistas.

3. **Búsqueda local inter-ruta (2-opt inter-ruta).** La búsqueda local implementada opera únicamente a nivel intra-ruta. Incorporar 2-opt inter-ruta, que permite intercambiar clientes entre rutas distintas, actuaría directamente sobre uno de los dos factores que explican la brecha de ACO con respecto al óptimo: el orden de asignación de clientes a vehículos (véase la Sección 3.2.2).

4. **Paralelización de la construcción de soluciones en ACO.** Las hormigas actualmente construyen soluciones de forma secuencial. Paralelizar esta etapa permitiría explorar un mayor número de iteraciones en el mismo tiempo de cómputo, con potencial de reducir el gap sin incrementar el límite de tiempo, aprovechando los núcleos disponibles que ACO actualmente deja ociosos.

5. **Enfoque híbrido ACO–MILP.** Dado que ACO ya identifica correctamente la cantidad de vehículos a activar, un paso natural es fijar ese subconjunto de vehículos como restricción para MILP. Con el espacio de búsqueda reducido, el solver exacto podría garantizar optimalidad en instancias que de otro modo resultarían inabordables, combinando la escalabilidad de ACO con la precisión de la programación entera mixta.

6. **Optimizaciones del modelo MILP.** Actualmente se utiliza la configuración por defecto de Pyomo para cada solver, sin ajustes adicionales. Explorar técnicas de preprocesamiento, poda de arcos imposibles según ventanas de tiempo y capacidades, o descomposición del modelo, podría ampliar el rango de instancias resolubles en tiempos razonables (véase la Sección 3.3.2).

7. **Exploración de funciones objetivo alternativas.** El modelo actual minimiza el costo total de transporte en términos de distancia recorrida. Sin embargo, distintos contextos operativos pueden priorizar objetivos diferentes: garantizar una distribución equitativa de carga de trabajo entre los transportistas, o minimizar el tiempo de finalización del último viaje para completar todos los envíos lo antes posible. Estas alternativas implican no solo cambios en la función objetivo, sino potencialmente nuevas restricciones que reflejen los criterios de negocio relevantes.

Apéndice A

Resultados completos de las ejecuciones

A.1. TWNN

Cuadro A.1: Resultados de TWNN sobre las 30 instancias.

c	v	Generador	Factible	Costo	Tiempo (s)	CPU (%)	RAM (GB)
5	10	Aleatorio	Sí	82	0.0001	0.0	0.094
10	10	Constructivo	Sí	209	0.0001	0.0	0.094
10	20	Constructivo	Sí	182	0.0002	0.0	0.094
15	15	Constructivo	Sí	298	0.0002	0.0	0.095
15	30	Aleatorio	No	–	0.0003	0.0	0.094
20	10	Aleatorio	No	–	0.0002	0.0	0.094
20	20	Aleatorio	No	–	0.0005	0.0	0.094
20	40	Constructivo	Sí	361	0.0006	0.0	0.095
25	25	Aleatorio	Sí	488	0.0007	0.0	0.094
25	50	Constructivo	Sí	431	0.0008	0.0	0.095
30	15	Constructivo	Sí	439	0.0005	0.0	0.095
30	30	Aleatorio	No	–	0.0004	0.0	0.095
30	60	Aleatorio	No	–	0.0011	0.0	0.095
35	35	Constructivo	Sí	697	0.0005	0.0	0.095
35	70	Constructivo	Sí	537	0.0015	0.0	0.096
40	20	Constructivo	Sí	519	0.0007	0.0	0.095
40	40	Aleatorio	No	–	0.0007	0.0	0.095
40	80	Aleatorio	No	–	0.0018	0.0	0.097
45	45	Aleatorio	Sí	845	0.0009	0.0	0.095
50	25	Aleatorio	No	–	0.0008	0.0	0.095
50	50	Constructivo	Sí	979	0.0011	0.0	0.096
55	55	Constructivo	Sí	1073	0.0013	0.0	0.096
60	30	Constructivo	Sí	748	0.0013	0.0	0.095
60	60	Constructivo	Sí	1266	0.0015	0.0	0.096
65	65	Aleatorio	No	–	0.0017	0.0	0.097
70	35	Aleatorio	No	–	0.0015	0.0	0.095
70	70	Constructivo	Sí	1376	0.0020	0.0	0.097
75	75	Constructivo	Sí	1400	0.0022	0.0	0.098

continúa en la página siguiente

(continuación — TWNN)

c	v	Generador	Factible	Costo	Tiempo (s)	CPU (%)	RAM (GB)
80	40	Aleatorio	No	–	0.0019	0.0	0.096
80	80	Aleatorio	No	–	0.0025	0.0	0.098

A.2. ACO

Cuadro A.2: Resultados de ACO sobre las 30 instancias.

c	v	Generador	Factible	Costo	Tiempo (s)	CPU (%)	RAM (GB)
5	10	Aleatorio	Sí	51	80.00	100.0	0.094
10	10	Constructivo	Sí	64	80.00	100.0	0.095
10	20	Constructivo	Sí	58	80.01	100.0	0.095
15	15	Constructivo	Sí	82	80.01	100.0	0.095
15	30	Aleatorio	Sí	108	80.00	100.0	0.095
20	10	Aleatorio	Sí	138	80.01	100.0	0.094
20	20	Aleatorio	Sí	121	80.02	100.0	0.095
20	40	Constructivo	Sí	91	80.03	100.0	0.095
25	25	Aleatorio	Sí	166	80.03	100.0	0.094
25	50	Constructivo	Sí	110	80.03	100.0	0.095
30	15	Constructivo	Sí	134	80.02	100.0	0.095
30	30	Aleatorio	Sí	191	80.02	100.0	0.095
30	60	Aleatorio	Sí	216	80.04	100.0	0.095
35	35	Constructivo	Sí	134	80.06	100.0	0.095
35	70	Constructivo	Sí	158	80.06	100.0	0.096
40	20	Constructivo	Sí	153	80.05	100.0	0.095
40	40	Aleatorio	Sí	260	80.02	100.0	0.095
40	80	Aleatorio	Sí	282	80.07	100.0	0.097
45	45	Aleatorio	Sí	307	80.05	100.0	0.095
50	25	Aleatorio	Sí	337	80.03	100.0	0.095
50	50	Constructivo	Sí	198	80.13	100.0	0.096
55	55	Constructivo	Sí	217	80.10	100.0	0.096
60	30	Constructivo	Sí	245	80.12	100.0	0.095
60	60	Constructivo	Sí	232	80.15	100.0	0.096
65	65	Aleatorio	No	–	80.07	100.0	0.097
70	35	Aleatorio	No	–	80.03	100.0	0.095
70	70	Constructivo	Sí	251	80.18	100.0	0.097
75	75	Constructivo	Sí	296	80.25	99.9	0.098
80	40	Aleatorio	No	–	80.07	100.0	0.096
80	80	Aleatorio	No	–	80.05	100.0	0.098

A.3. Gurobi

Cuadro A.3: Resultados de Gurobi sobre las 30 instancias.

c	v	Generador	Factible	Costo	Tiempo (s)	CPU (%)	RAM (GB)
5	10	Aleatorio	Sí	51	1.46	31.6	0.130
10	10	Constructivo	Sí	50	0.67	148.6	0.159
10	20	Constructivo	Sí	49	4.06	85.7	0.262
15	15	Constructivo	Sí	65	2.33	187.4	0.253
15	30	Aleatorio	Sí	94	61.65	486.2	0.943
20	10	Aleatorio	Sí	118	6.72	650.6	0.320
20	20	Aleatorio	Sí	107	757.31	617.8	1.281
20	40	Constructivo	Sí	70	34.92	242.1	1.440
25	25	Aleatorio	Sí	119	1214.99	710.1	2.335
25	50	Constructivo	Sí	80	71.70	240.8	2.567
30	15	Constructivo	Sí	103	200.51	665.3	1.133
30	30	Aleatorio	Sí	140	1215.85	721.7	2.990
30	60	Aleatorio	Sí	144	257.33	365.5	4.364
35	35	Constructivo	Sí	103	85.44	354.4	1.981
35	70	Constructivo	Sí	106	534.39	357.3	6.540
40	20	Constructivo	Sí	120	1207.64	713.7	2.671
40	40	Aleatorio	Sí	157	1235.53	661.7	4.148
40	80	Aleatorio	No	–	290.97	206.8	9.833
45	45	Aleatorio	Sí	185	1249.12	610.9	5.619
50	25	Aleatorio	Sí	200	1218.37	696.8	3.618
50	50	Constructivo	Sí	143	1267.24	552.8	7.475
55	55	Constructivo	Sí	145	1288.21	562.3	7.136
60	30	Constructivo	Sí	196	1225.90	678.1	4.283
60	60	Constructivo	Sí	165	1315.88	522.7	8.540
65	65	Aleatorio	Sí	270	1346.25	338.4	9.151
70	35	Aleatorio	No	–	1238.63	580.5	6.246
70	70	Constructivo	No	–	181.93	132.2	9.803
75	75	Constructivo	No	–	214.97	109.4	10.243
80	40	Aleatorio	No	–	1257.20	501.1	8.562
80	80	Aleatorio	No	–	300.57	101.9	9.950

A.4. HiGHS

Cuadro A.4: Resultados de HiGHS sobre las 30 instancias.

c	v	Generador	Factible	Costo	Tiempo (s)	CPU (%)	RAM (GB)
5	10	Aleatorio	Sí	51	0.87	105.4	0.130
10	10	Constructivo	Sí	50	1.70	105.6	0.173
10	20	Constructivo	Sí	48	6.75	102.2	0.378
15	15	Constructivo	Sí	65	8.41	103.5	0.367
15	30	Aleatorio	Sí	94	1220.51	139.6	1.291

continúa en la página siguiente

(continuación — HiGHS)

c	v	Generador	Factible	Costo	Tiempo (s)	CPU (%)	RAM (GB)
20	10	Aleatorio	Sí	118	399.00	111.6	0.600
20	20	Aleatorio	Sí	107	1213.90	101.9	1.106
20	40	Constructivo	Sí	70	89.42	101.6	2.250
25	25	Aleatorio	Sí	134	1217.31	100.7	1.507
25	50	Constructivo	Sí	87	1290.48	100.9	5.050
30	15	Constructivo	Sí	118	1206.38	101.5	1.262
30	30	Aleatorio	Sí	162	1229.15	100.8	2.390
30	60	Aleatorio	Sí	144	1361.66	101.8	7.156
35	35	Constructivo	Sí	168	1244.22	101.5	3.975
35	70	Constructivo	No	–	283.95	103.3	9.772
40	20	Constructivo	Sí	180	1214.76	102.6	2.815
40	40	Aleatorio	Sí	179	1267.39	102.9	4.472
40	80	Aleatorio	No	–	259.95	100.0	9.766
45	45	Aleatorio	Sí	313	1299.25	103.7	6.258
50	25	Aleatorio	Sí	295	1228.10	103.0	3.533
50	50	Constructivo	Sí	222	1330.65	105.2	7.981
55	55	Constructivo	No	–	1142.12	105.7	9.775
60	30	Constructivo	Sí	328	1249.78	104.4	5.211
60	60	Constructivo	No	–	298.00	112.3	9.788
65	65	Aleatorio	No	–	278.99	100.0	10.716
70	35	Aleatorio	No	–	1277.29	109.6	6.776
70	70	Constructivo	No	–	245.94	100.0	9.781
75	75	Constructivo	No	–	219.08	100.0	9.792
80	40	Aleatorio	No	–	1311.55	110.8	9.330
80	80	Aleatorio	No	–	189.04	100.0	9.795

Bibliografía

[1] Dantzig, G. B. y Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91.

[2] Garey, M. R. y Johnson, D. S. (1990). *A Guide to the Theory of NP-Completeness. Computers and Intractability*, pp. 37–79.

[3] Kallehauge, B., Larsen, J., Madsen, O. B. y Solomon, M. M. (2005). Vehicle routing problem with time windows. En *Column Generation* (pp. 67–98). Boston, MA: Springer US.

[4] Dantzig, G. B. (1990). Origins of the simplex method. En *A History of Scientific Computing* (pp. 141–151).

[5] Ceria, S., Cordier, C., Marchand, H. y Wolsey, L. A. (1998). Cutting planes for integer programs with general integer variables. *Mathematical Programming*, 81(2), 201–214.

[6] Bynum, M. L., Hackebeil, G. A., Hart, W. E., Laird, C. D., Nicholson, B. L., Siirola, J. D., Watson, J.-P. y Woodruff, D. L. (2021). *Pyomo – Optimization Modeling in Python*, 3.ª ed., vol. 67. Springer.

[7] Hart, W. E., Watson, J.-P. y Woodruff, D. L. (2011). Pyomo: Modeling and solving mathematical programs in Python. *Mathematical Programming Computation*, 3(3), 219–260.

[8] Huangfu, Q. y Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119–142. doi:10.1007/s12532-017-0130-5

[9] Gurobi Optimization, LLC. (2026). *Gurobi Optimizer Reference Manual*. `https://www.gurobi.com`

[10] Forrest, J. y Lougee-Heimer, R. (2005). CBC User Guide. En *Emerging Theory, Methods, and Applications*, pp. 257–277. doi:10.1287/educ.1053.0020

[11] Makhorin, A. (2011). *GLPK (GNU Linear Programming Kit) v4.47.1*. Moscow: Department for Applied Informatics, Moscow Aviation Institute.

[12] Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2), 254–265.

[13] Ozaki, Y., Watanabe, S. y Yanase, T. (2025). OptunaHub: A Platform for Black-Box Optimization. *arXiv preprint arXiv:2510.02798*.

[14] Yadan, O. (2019). *Hydra – A framework for elegantly configuring complex applications*. Github. `https://github.com/facebookresearch/hydra`

[15] Clarke, G. y Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581.

[16] Dorigo, M. y Stützle, T. (2004). *Ant Colony Optimization*. Cambridge, MA: MIT Press.

[17] Gracia, C., Diezma-Iglesias, B. y Barreiro, P. (2013). A hybrid genetic algorithm for route optimization in the bale collecting problem. *Spanish Journal of Agricultural Research*, 11(3), 603–614. doi:10.5424/sjar/2013113-3635

[18] Laporte, G. (2009). Fifty years of vehicle routing. *Transportation Science*, 43(4), 408–416. doi:10.1287/trsc.1090.0301

[19] Polacek, M., Hartl, R. F., Doerner, K. y Reimann, M. (2004). A variable neighborhood search for the multi depot vehicle routing problem with time windows. *Journal of Heuristics*, 10(6), 613–627. doi:10.1007/s10732-005-5891-3

[20] Bochtis, D. D. y Sørensen, C. G. (2010). The vehicle routing problem in field logistics: Part II. *Biosystems Engineering*, 105(2), 180–188. doi:10.1016/j.biosystemseng.2009.10.006

[21] Bakhtiari, A. A., Navid, H., Mehri, J. y Bochtis, D. D. (2011). Optimal route planning of agricultural field operations using ant colony optimization. *Agricultural Engineering International: CIGR Journal*, 13(4).

[22] Khachiyan, L. G. (1979). A polynomial algorithm in linear programming. *Doklady Akademii Nauk*, 244(5), 1093–1096.

[23] Achterberg, T., Bixby, R. E., Gu, Z., Rothberg, E. y Weninger, D. (2014). Multi-row presolve reductions in mixed integer programming. En *Proceedings of the 26th Annual INFORMS Computing Society Conference*, pp. 4–18.

[24] Zanetti, F. y Gondzio, J. (2025). A factorisation-based regularised interior point method using the augmented system. *arXiv preprint arXiv:2508.04370*.

[25] Abreu, L. R. (2026). Mixed-integer versus constraint programming solvers: An extensive comparison with the Job Shop Scheduling Problem. *Proceeding Series of the Brazilian Society of Computational and Applied Mathematics*, 12(1), 1–7.

Certificación de la versión final

Los abajo firmantes certifican que la versión final del presente Trabajo Especial de Licenciatura en Ciencias de la Computación titulado *Asignación Óptima de Viajes para Transporte de Maquinaria Agrícola con Restricciones Temporales y Operativas*, presentado por Tomás Marmay, se corresponde con la versión aprobada por el tribunal evaluador.

_____

Tomás Marmay
Autor

_____

Miembro del tribunal

_____

Miembro del tribunal